



Department of Computer Systems Engineering
Faculty of Computing

Environment-Aware Service Discovery and Load Balancing for Kubernetes-Native Malware Inspection Infrastructure

An Individual Research Thesis

Submitted in partial fulfilment of the requirements for the degree of

BSc (Hons) in Information Technology specialising in Cyber Security

Name	IT Number
PANDUKABHAYA H D T	IT22345028

Group Project: CAPSLock — Kubernetes-native Malware Inspection and Policy Enforcement Platform

Project ID: 25-26J-043

Individual Component: Service Discovery and Load Balancing (SDLB)

Academic Year: 2025/2026

DECLARATION OF ORIGINALITY

I, Pandukabhaya H. D. T., hereby declare that this thesis, submitted in partial fulfilment of the requirements for the degree of BSc (Hons) in Information Technology specialising in Cyber Security, is entirely the result of my own independent investigation and research, except where indicated by reference. The content reported here has not been submitted, either in whole or in part, for any other degree, diploma or qualification at this or any other university or institute of higher learning. All sources of information used in this dissertation have been duly acknowledged in accordance with the academic referencing conventions of the Sri Lanka Institute of Information Technology.

The individual component described in this thesis, the Service Discovery and Load Balancing (SDLB) module, forms one of four research contributions within the CAPSLock group research project (Project ID 25-26J-043). The author was solely responsible for the design, implementation, experimental evaluation, and documentation of the SDLB component. References to the other three components of the platform, namely the Multi-Environment Deployment System (MEDS), the Environment-Aware Policy Engine (EAPE), and the ICAP Operator, are included only where required to establish architectural context and integration boundaries; the originality of this thesis is confined to the SDLB component.

Signed: _____

Name: Pandukabhaya H. D. T.

Date: April 2026

ACKNOWLEDGEMENTS

The author wishes to express sincere gratitude to the supervisory staff of the Faculty of Computing at the Sri Lanka Institute of Information Technology for their guidance and feedback throughout the course of this research. Thanks are extended to the CAPSLock project group members for their collaborative spirit in establishing the shared architectural foundations upon which this individual component was built. The author also acknowledges the maintainers of the open-source tools and libraries upon which this work depends, including the Kubernetes, Istio, c-icap, ClamAV, and Prometheus communities, whose publicly documented engineering has made this research both tractable and reproducible. Finally, sincere appreciation is extended to family and friends for their patience and encouragement during the writing of this thesis.

ABSTRACT

Kubernetes has become the de facto substrate for modern container orchestration, yet the networking fabric it exposes remains uniform with respect to operational context. Service discovery and load balancing inside a Kubernetes cluster are typically applied with a single policy across every namespace, with no distinction between production, staging, and development workloads. In security-sensitive deployments, and in particular in clusters that perform inline malware inspection using the Internet Content Adaptation Protocol (ICAP), this uniformity becomes a structural liability: a routing configuration strict enough for production is unnecessarily punitive in development, whereas a configuration tuned for development leaves production inspection infrastructure vulnerable to unbalanced load and undetected endpoint failures.

This thesis presents the design, implementation, and empirical evaluation of the Service Discovery and Load Balancing (SDLB) component of the CAPSLock Kubernetes security platform. SDLB is a Kubernetes-native controller that consumes environment classification signals from an upstream policy engine and dynamically generates Istio VirtualService and DestinationRule configurations tailored to each environment tier. Three distinct routing policies are defined: strict health-gated round-robin for Production, weighted 50/30/20 routing for Staging, and permissive single-instance routing for Development. An adaptive decision engine sits above these base policies and reacts to traffic trend hysteresis, per-instance ICAP health degradation, cooldown windows, and predictive spread-mode transitions.

The component was evaluated on a three-node Minikube cluster running Kubernetes v1.29 with three ICAP replicas (icap-a, icap-b, icap-c) on TCP port 1344, fronted by the Istio ingress gateway. Across 2,400 routing-correctness requests, Production achieved a 33.8%/33.1%/33.1% distribution (inter-instance variance 0.7%, threshold <2.0%), Staging achieved 51.0%/29.3%/19.7% (maximum weight deviation $\pm 1.0\%$, threshold $\pm 2.0\%$), and Development achieved 97.6% primary-endpoint share (threshold >95.0%). Controlled failure injection produced a mean failure detection time of 29.7 seconds, a mean traffic redistribution time of 3.9 seconds, a mean recovery re-admission time of 11.2 seconds, and zero request loss during failover. A 10-minute sustained load test at 1,200 requests per minute processed 12,000 inspection requests at a 99.83% success rate with latency variation of ± 4 ms. Mean service-mesh routing overhead was 7.99%, classified as Good against the $\leq 10\%$ acceptable threshold. Scalability measurements recorded an 11.0% latency reduction from one to three ICAP instances. Eight canonical decision-engine scenarios were validated against the reference state machine.

The thesis contributes: (i) an environment-aware routing framework that unifies service discovery, policy-driven load balancing, and compliance audit logging within a single Kubernetes controller; (ii) an adaptive decision engine with health-penalty-weighted effective-rate scoring; and (iii) PCI-DSS v4.0 and CIS Kubernetes Benchmark v1.9 compliance mappings for every routing state transition. The result is a low-overhead, environment-aware routing layer that is demonstrably suitable for production-grade malware inspection workloads.

Keywords: Kubernetes, Istio, service discovery, load balancing, ICAP, malware inspection, environment-aware routing, service mesh, cloud-native security.

LIST OF FIGURES

Figure 2.1 — High-level architecture of SDLB within the CAPSLock platform.

Figure 2.2 — Interaction between EAPE classification signals and SDLB routing configuration generation.

Figure 2.3 — Istio VirtualService and DestinationRule object model produced by the SDLB controller.

Figure 2.4 — Routing decision state machine for single-mode and spread-mode transitions.

Figure 2.5 — Inspection traffic flow through the CAPSLock pipeline.

Figure 2.6 — Minikube experimental topology with three ICAP replicas and the Istio ingress gateway.

Figure 3.1 — Observed Production round-robin distribution across 1,000 inspection requests.

Figure 3.2 — Observed Staging weighted distribution against the 50/30/20 target.

Figure 3.3 — Failure detection, redistribution, and recovery timing across three failover tests.

Figure 3.4 — Latency breakdown showing routing overhead relative to ICAP inspection time.

Figure 3.5 — Scalability curve from one to three ICAP instances.

Figure 3.6 — Eight canonical decision-engine scenarios from the live dashboard.

LIST OF TABLES

Table 1.1 — Summary of related work and its relationship to SDLB’s contributions.

Table 2.1 — SDLB controller configuration parameters and default values.

Table 2.2 — Routing state-machine decision hierarchy.

Table 2.3 — Experimental environment specification.

Table 2.4 — PCI-DSS v4.0 and CIS control mappings applied to SDLB events.

Table 3.1 — Production round-robin observed distribution.

Table 3.2 — Staging weighted routing observed distribution.

Table 3.3 — Development permissive routing observed distribution.

Table 3.4 — Failover test results.

Table 3.5 — Latency breakdown across 12,000 requests.

Table 3.6 — Scalability results from one to three ICAP instances.

Table 3.7 — Continuous load stability results over ten minutes.

Table 3.8 — Latency comparison across environment tiers.

Table 3.9 — Canonical decision-engine scenarios.

Table 3.10 — Consolidated benchmark summary.

LIST OF ABBREVIATIONS

API	— Application Programming Interface
CAPSLock	— Container-Aware Policy and Security Lockdown (group project)
CIS	— Centre for Internet Security
CNCF	— Cloud Native Computing Foundation
CRD	— Custom Resource Definition
DLP	— Data Loss Prevention
EAPE	— Environment-Aware Policy Engine (CAPSLock component 2)
HPA	— Horizontal Pod Autoscaler
HTTP	— Hypertext Transfer Protocol
ICAP	— Internet Content Adaptation Protocol (RFC 3507)
IETF	— Internet Engineering Task Force
JSON	— JavaScript Object Notation
mTLS	— Mutual Transport Layer Security
MEDS	— Multi-Environment Deployment System (CAPSLock component 1)
NIST	— National Institute of Standards and Technology
OPA	— Open Policy Agent
PCI-DSS	— Payment Card Industry Data Security Standard
RBAC	— Role-Based Access Control
REQMOD	— ICAP Request Modification mode
RESPMOD	— ICAP Response Modification mode
RFC	— Request for Comments
SDLB	— Service Discovery and Load Balancing (CAPSLock component 3; subject of this thesis)
SLIIT	— Sri Lanka Institute of Information Technology
TCP	— Transmission Control Protocol
XML	— Extensible Markup Language
YAML	— YAML Ain't Markup Language

TABLE OF CONTENTS

DECLARATION OF ORIGINALITY	2
ACKNOWLEDGEMENTS	3
ABSTRACT	5
LIST OF FIGURES	6
LIST OF TABLES	7
LIST OF ABBREVIATIONS	8
TABLE OF CONTENTS	9
CHAPTER 1 — INTRODUCTION	12
1.1 Background and Motivation.....	12
1.2 Literature Review	13
1.2.1 Kubernetes Service Discovery and Networking	13
1.2.2 Service Mesh and Traffic Management.....	14
1.2.3 Health Checking and Adaptive Load Balancing.....	14
1.2.4 Cloud-Native Security and Zero Trust	15
1.2.5 ICAP Protocol and Content Inspection	15
1.2.6 Synthesis.....	15
1.3 Research Gap.....	16
1.4 Research Problem Statement.....	17
1.5 Research Objectives.....	17
1.6 Scope and Limitations.....	18
1.7 Chapter Overview.....	18
CHAPTER 2 — METHODOLOGY	20
2.1 Overall SDLB Architecture within CAPSLock	20
2.2 Environment-Aware Routing Design	22
2.2.1 Environment Classification Interface.....	22
2.2.2 Production Routing Policy: Round-Robin with Health Gating.....	22
2.2.3 Staging Routing Policy: Weighted 50/30/20 Routing	23
2.2.4 Development Routing Policy: Permissive Single-Instance Routing.....	23
2.3 SDLB Controller Design and Implementation.....	24
2.3.1 Kubernetes Controller Architecture	24
2.3.2 Service Discovery Mechanism.....	25
2.3.3 Health Monitoring Subsystem.....	26
2.3.4 Routing Configuration Generation	26
2.4 Adaptive Routing Decision Engine.....	27
2.4.1 Traffic Spike Detection and Spread Mode.....	27

2.4.2 ICAP Health Scoring and Penalty Logic	28
2.4.3 Predictive Failover Mechanism.....	28
2.4.4 Cooldown and Recovery Logic.....	29
2.4.5 Routing Decision State Machine	29
2.5 Inspection Traffic Flow Through the CAPSLock Pipeline.....	30
2.6 Experimental Environment Setup	31
2.6.1 Minikube Cluster Configuration.....	31
2.6.2 ICAP Instance Deployment	32
2.6.3 Istio Service Mesh Configuration	32
2.6.4 Traffic Generation Methodology.....	33
2.7 Tools, Technologies, and Frameworks	33
2.8 Compliance Framework Alignment	34
2.9 Commercialisation Aspects	35
2.9.1 Market Analysis and Positioning.....	35
2.9.2 Revenue and Business Model	35
2.9.3 Competitive Advantages	36
CHAPTER 3 — RESULTS AND DISCUSSION	37
3.1 Routing Policy Correctness Evaluation	37
3.1.1 Production Round-Robin Accuracy	37
3.1.2 Staging Weighted Routing Accuracy	38
3.1.3 Development Permissive Routing Accuracy	39
3.2 Health Monitoring and Failover Evaluation	39
3.2.1 Failure Detection Time	40
3.2.2 Traffic Redistribution Time	40
3.2.3 Instance Recovery Re-Admission.....	40
3.2.4 Request Loss During Failover.....	41
3.3 System Overhead Benchmark	41
3.3.1 Routing Overhead Percentage.....	41
3.3.2 Latency Breakdown.....	42
3.4 Scalability Benchmark	43
3.4.1 Single-Instance Baseline (118 ms)	43
3.4.2 Two-Instance Configuration (109 ms, 7.6% Improvement)	43
3.4.3 Three-Instance Configuration (105 ms, 11.0% Improvement)	43
3.4.4 Instance Detection Time (3–5 Seconds)	44
3.5 Continuous Load Stability	44
3.5.1 Ten-Minute Sustained Test (12,000 Requests at 1,200/min).....	44
3.5.2 Success Rate, Latency Variation, and Routing Errors	45

3.6 Latency Comparison Across Environment Tiers	45
3.6.1 Production (120 ms), Staging (~112 ms), Development (95 ms)	45
3.6.2 Development Routing Latency Advantage (20.8% Improvement)	46
3.7 Adaptive Routing Decision Scenarios	46
3.8 Consolidated Benchmark Summary	48
3.9 Discussion	48
3.9.1 Interpreting the Results	48
3.9.2 Hardships and Iterative Improvements Encountered	49
3.9.3 Comparison with Related Work	49
3.9.4 Implications for Cloud-Native Security Infrastructure	50
3.9.5 Future Research Directions	50
CHAPTER 4 — CONCLUSION	52
4.1 Summary of Research Contributions	52
4.2 Achievement of Research Objectives	52
4.3 Practical Implications	53
4.4 Limitations and Future Work	54
4.5 Closing Remarks	54
REFERENCES	56

CHAPTER 1 — INTRODUCTION

1.1 Background and Motivation

Container orchestration has fundamentally reshaped the operational model of modern distributed systems. Released by Google in 2014 and subsequently adopted as the flagship project of the Cloud Native Computing Foundation, Kubernetes has emerged as the dominant substrate for containerised workload management. Its declarative object model, controller-based reconciliation loop, and pluggable networking layer have produced a computing environment that is both highly automated and highly dynamic, with ephemeral workloads arriving and departing at rates that would have been unthinkable under traditional virtualisation. That dynamism, however, introduces a corresponding operational challenge: every service inside a Kubernetes cluster must be discoverable, addressable, and reachable through a routing layer capable of accommodating continuous change without operator intervention.

Service discovery and load balancing are therefore foundational concerns in any non-trivial Kubernetes deployment. The platform’s native discovery mechanism, DNS resolution backed by EndpointSlices synchronised from Service objects, is a general-purpose solution that treats every workload identically. For most workloads that uniformity is acceptable. Once inline security enforcement is introduced, however, the picture changes substantially. A security gateway that performs content inspection must route every request through a specialised scanning backend, and the performance characteristics and reliability guarantees of that backend are material to both the user experience and the security posture of the platform. An inspection backend that is overloaded delivers slow responses and may silently drop scans; one that is underloaded wastes expensive compute. A routing layer that cannot distinguish between environments will either over-inspect development traffic and frustrate engineers, or under-inspect production traffic and compromise the security contract that the platform exists to enforce.

The CAPSLock project (Project ID 25-26J-043) addresses this tension by introducing a four-component Kubernetes-native security platform in which malware inspection, policy enforcement, environment classification, and deployment promotion are implemented as first-class cluster primitives. CAPSLock is organised around the principle that security controls must be proportional to the operational sensitivity of the workloads they protect. Within this architecture, the Service Discovery and Load Balancing (SDLB) component, which is the subject of this individual thesis, is responsible for the routing fabric that carries inspection traffic from application workloads to the ICAP-based scanning backends managed by the ICAP Operator. The central question SDLB is designed to answer is: how should inspection traffic be routed when different namespaces in the same cluster are subject to different security regimes?

The motivation for treating service discovery and load balancing as a security-sensitive concern, rather than a purely networking one, is grounded in three observations drawn from the cloud-native security literature. First, lateral movement inside flat Kubernetes networks has been empirically documented as a significant attack surface, and service-mesh-level routing is one of the few enforcement points at which per-request decisions can be made transparently to application code [4]. Second, the Internet Content Adaptation Protocol, standardised in RFC 3507 [17], is the enterprise-standard mechanism for offloading content scanning from proxies to dedicated inspection services, yet it has no native Kubernetes integration and is rarely wired into a cluster’s routing layer as a first-class citizen. Third,

compliance frameworks such as PCI-DSS v4.0 [9] and the CIS Kubernetes Benchmark v1.9 [8] require auditable logging of access decisions and routing changes, and these events must be produced deterministically by whichever component makes the decision, which in CAPSLock’s case is SDLB itself.

Given this problem framing, SDLB occupies a distinctive position within the CAPSLock platform. It is not a generic service mesh, nor a generic Kubernetes controller; rather, it is an environment-aware controller that mediates between the policy signals emitted by the Environment-Aware Policy Engine (EAPE), the health state of the ICAP backends managed by the ICAP Operator, and the traffic management primitives exposed by the Istio service mesh. At this intersection, SDLB is uniquely positioned to make routing decisions that are simultaneously aware of the environment tier of the source workload, the health trajectory of each inspection backend, and the compliance regime under which the resulting audit events must be logged. The remainder of this introduction situates SDLB within the broader research landscape, articulates the specific research gap the component addresses, formalises the research question, and enumerates the objectives against which the component was evaluated.

1.2 Literature Review

The literature relevant to SDLB spans four broad domains: Kubernetes-native service discovery and networking; service mesh architectures and traffic management; cloud-native load balancing and health checking; and the ICAP protocol in enterprise content-inspection deployments. Each of these bodies of literature has developed largely independently, and one of the contributions of the present work is to demonstrate that environment-aware routing for security workloads requires their integration rather than their isolated application.

1.2.1 Kubernetes Service Discovery and Networking

Burns et al. [21] provided the canonical description of the Kubernetes platform and its reconciliation model, tracing the platform’s lineage from the internal Borg and Omega systems at Google. Their paper established the desired-state reconciliation pattern on which every subsequent Kubernetes controller, including the SDLB controller presented here, is based. Hausenblas and Schimanski [19] and Dobies and Wood [20] subsequently codified the Operator pattern, in which a domain-specific controller watches Custom Resource Definitions (CRDs) and performs reconciliation actions against both the Kubernetes API and external systems. SDLB is implemented as a controller rather than a full Operator, because the authoritative resources it manipulates, namely Istio VirtualService and DestinationRule objects, are not owned by SDLB itself; nonetheless, the reconciliation-loop discipline advocated by Hausenblas and Schimanski is observed throughout the controller’s design.

Shamim et al. [1] provide the most widely cited systematisation of Kubernetes security practices. Their work decomposes the security landscape into eleven domains and explicitly identifies the coordination between authorisation, admission control, network policy, and pod security as an unsolved operational problem. Minna et al. [4] extend this analysis to the networking layer, demonstrating that Kubernetes’s flat-by-default network creates significant lateral movement risk that cannot be mitigated by admission controllers alone. This finding directly motivates the use of a service mesh, rather than raw Kubernetes Services, as the routing enforcement point in CAPSLock. Sultan et al. [3] and Rice [10] provide complementary analyses of container security issues, reinforcing the observation

that most production deployments rely on security-unaware load balancers that cannot make per-request routing decisions based on workload context. Merkel [30] traces the origins of container-based deployment from Linux namespaces through Docker, establishing the technological lineage from which Kubernetes evolved.

Rahman et al. [2] contribute the most operationally relevant empirical study to date on misconfiguration in Kubernetes manifests, documenting systemic gaps in security-related configuration across open-source deployments. Their dataset motivates the design choice in SDLB to accept ambiguous or partially labelled namespaces as legitimate inputs from EAPE, rather than relying exclusively on well-formed environment labels. Cesarano and Natella [6] further demonstrate that admission control alone is insufficient for field-level policy enforcement, supporting the case for routing-layer enforcement carried out by SDLB in concert with upstream admission controls. Sissodiya et al. [5] extend this line of inquiry through formal verification methods, confirming that policy misconfiguration in RBAC rules is a persistent structural problem that static tooling alone cannot resolve.

1.2.2 Service Mesh and Traffic Management

Li et al. [15] surveyed the service mesh as an architectural pattern comprehensively, identifying service discovery, traffic management, observability, and security as the four primary problem domains addressed by mesh implementations. Their taxonomy characterises a mesh as a dedicated data plane of sidecar proxies coordinated by a centralised control plane; this separation allows routing policy to be applied transparently to application code, and it is exactly this property that SDLB exploits when generating VirtualService and DestinationRule configurations. NIST SP 800-204A, authored by Chandramouli and Butcher [16], formalises security guidance for service meshes in microservices deployments, endorsing Istio as the reference platform and recommending mutual TLS enforcement, non-bypassable policy enforcement, and secure service discovery as baseline controls. SDLB inherits all three properties from the Istio data plane without implementing them directly.

The most directly relevant empirical study of Istio traffic management is by Saleh Sedghpour et al. [13], who evaluated circuit breaking and retry policies through controlled Kubernetes testbed experiments. Their finding that policies calibrated for production throughput introduce unacceptable latency in development environments is precisely the observation that SDLB’s environment-aware design is intended to address; this thesis explicitly replicates their testbed methodology in the evaluation reported in Chapter 3. Hahn et al. [14] evaluate security vulnerabilities in Consul, Istio, and Linkerd under default and optimally configured scenarios, exposing design flaws present even in recommended configurations. Their findings imply that service mesh security must be augmented by admission-level controls, a conclusion consistent with CAPSLock’s overall architecture in which EAPE, SDLB, and the ICAP Operator each enforce a portion of the total security contract.

1.2.3 Health Checking and Adaptive Load Balancing

Nguyen et al. [22] analysed horizontal pod autoscaling in Kubernetes, finding that the platform’s default reliance on CPU and memory metrics fails to capture application-level demand for workloads whose primary consumption is not CPU-bound. For a ClamAV-backed ICAP service, the operationally significant signals are scan throughput, signature freshness, and error rate, none of which are visible to the default HPA. SDLB’s health scoring inputs, supplied by the ICAP Operator, address this gap by surfacing scan-layer metrics as a first-class routing signal.

Classical load-balancing literature has long distinguished between oblivious algorithms such as round-robin and random selection, and adaptive algorithms that incorporate backend state. SDLB’s decision engine combines the simplicity of oblivious round-robin, applied as the Production base policy, with a lightweight adaptive layer that overrides base behaviour only when health or traffic-trend signals warrant intervention. This hybrid design reflects observations in the service mesh literature that fully adaptive algorithms can introduce oscillation, particularly in the presence of health-signal noise, and that hysteresis thresholds are necessary to suppress spurious mode changes [13], [15]. Liu et al. [28] provide further empirical grounding for health-aware load balancing in service mesh architectures, demonstrating that health score weighting significantly reduces routing to degraded instances compared with purely reactive circuit-breaking approaches.

1.2.4 Cloud-Native Security and Zero Trust

The CNCF Cloud Native Security Whitepaper [23] defines the cloud-native security lifecycle in four phases (Develop, Distribute, Deploy, Runtime) and identifies continuous content inspection as a Runtime-phase requirement. Rose et al. [25], in NIST SP 800-207, formalise the Zero Trust Architecture and establish ‘never trust, always verify’ as the governing principle for multi-tenant environments such as shared Kubernetes clusters. D’Silva and Ambawade [24] provide a concrete Kubernetes zero-trust implementation and demonstrate that microsegmentation combined with admission control produces effective isolation. SDLB participates in this zero-trust model by ensuring that every inspection request traverses a verified healthy endpoint and that every routing decision is recorded as an auditable event with PCI-DSS and CIS Benchmark mappings.

1.2.5 ICAP Protocol and Content Inspection

Elson and Cerpa [17] specified the Internet Content Adaptation Protocol in IETF RFC 3507 as a lightweight HTTP-like protocol for executing remote procedure calls on HTTP messages. Two primary interaction modes are defined: REQMOD for request-side adaptation, covering antivirus scanning, URL filtering, and authentication, and RESPMOD for response-side adaptation, covering DLP, content filtering, and data transformation. ICAP is well established in enterprise proxy architectures, with Symantec, McAfee, Sophos, and F5 all implementing ICAP client interfaces, but it has no Kubernetes-native management mechanism. Wong et al. [18] identify malicious content injection through container network interfaces as a significant unmitigated threat in containerised deployments, yet no prior work bridges ICAP content inspection with Kubernetes orchestration as a unified security control. The combined effort of the ICAP Operator and SDLB within CAPSLock is intended to close precisely this gap.

1.2.6 Synthesis

Table 1.1 summarises how each strand of the literature relates to specific contributions claimed by the present thesis. A key structural observation becomes explicit in the table: the five research contributions of SDLB, namely environment-aware routing, ICAP-native health probing, adaptive decision logic, compliance event mapping, and testbed evaluation methodology, each draw on a distinct body of prior work, and none of the prior works unifies all five. It is their integration that constitutes the novel contribution of this thesis.

Domain	Representative Work	Relationship to SDLB
Kubernetes reconciliation model	Burns et al. [21]; Hausenblas & Schimanski [19]	Controller design pattern inherited
Kubernetes security landscape	Shamim et al. [1]; Minna et al. [4]	Motivation for mesh-based routing enforcement
Misconfiguration empirics	Rahman et al. [2]	Tolerance for partial namespace labels
Service mesh taxonomy	Li et al. [15]; Chandramouli & Butcher [16]	Istio as routing substrate; security posture
Istio traffic management	Saleh Sedghpour et al. [13]	Environment-specific policy tuning; testbed methodology
Mesh security pitfalls	Hahn et al. [14]	Need for admission-layer complement
Health-aware load balancing	Liu et al. [28]; Nguyen et al. [22]	ICAP health as routing signal; HPA limitations
Cloud-native security lifecycle	CNCF [23]	Runtime inspection requirement
Zero trust	Rose et al. [25]; D'Silva & Ambawade [24]	Continuous endpoint verification model
ICAP specification	Elson & Cerpa [17]	Backend protocol and health-probe method
Container attack surface	Wong et al. [18]; Sultan et al. [3]	Motivation for inline inspection

Table 1.1 — Summary of related work and its relationship to SDLB's contributions.

1.3 Research Gap

Three gaps in the existing literature motivate the SDLB component.

First, environment-aware routing remains underexplored. Saleh Sedghpour et al. [13] demonstrate that service mesh traffic management policies require per-environment tuning, but they do not propose an operational mechanism for making that tuning automatic. In practice, production engineering teams configure Istio by hand and apply the same configuration uniformly across namespaces. To the knowledge of this author, no prior published system automatically derives routing policy from environment classification signals and applies it through service mesh primitives.

Second, ICAP is absent from Kubernetes-native research. Although Wong et al. [18] identify content inspection as a significant gap and the ICAP protocol [17] has been an industry standard for over two decades, the research literature contains no empirically evaluated design for integrating ICAP scanning backends into Kubernetes service discovery and routing. Existing work on containerised antivirus deployments treats scanning engines as opaque containers and relies on generic HTTP load balancers; this approach cannot exploit the ICAP OPTIONS protocol for fine-grained health probing, and it cannot bridge inspection outcomes into compliance audit logs.

Third, routing events are rarely mapped to compliance frameworks. PCI-DSS v4.0 [9] requires auditable logging of access decisions (Requirements 10.2.1 through 10.6.1), and the CIS Kubernetes Benchmark v1.9 [8] enumerates specific controls (4.1.1, 4.2.1, 4.4.1) that presuppose deterministic logging of routing state transitions. Generic load balancers and general-purpose service meshes emit metrics and logs, but they do not annotate those

outputs with compliance framework mappings. SDLB closes this gap by producing compliance-annotated events for every routing state change.

Together, these three gaps define an opportunity for a routing system that is environment-aware, ICAP-native, and compliance-auditable. SDLB is designed to occupy exactly this niche.

1.4 Research Problem Statement

The research problem addressed by this thesis may be stated as follows: How can service discovery and load balancing for ICAP-based malware inspection infrastructure in a Kubernetes cluster be automated such that the routing policy applied to each inspection request is proportional to the environmental sensitivity of the source workload, reactive to the real-time health of the inspection backends, and recorded as a compliance-auditable event, while introducing overhead low enough to preserve the latency characteristics required of a production-grade security gateway?

This problem is decomposable into three operational sub-problems. First, how can environment classification signals, produced by an upstream engine that is not tightly coupled to the routing layer, be translated into concrete service mesh configurations within timescales acceptable to a cloud-native platform, on the order of seconds rather than minutes? Second, how can per-instance health signals from ICAP backends be aggregated and used to drive routing decisions that are stable, meaning non-oscillating, under realistic traffic patterns? Third, how can routing state transitions, whether triggered by environment change, health degradation, or traffic trend analysis, be emitted as audit events whose fields correspond to the controls demanded by PCI-DSS and CIS Kubernetes Benchmark requirements? By construction and empirical evaluation, this thesis demonstrates that affirmative answers exist for each of these sub-problems.

1.5 Research Objectives

The following five objectives, each amenable to empirical verification, guided the design and evaluation of SDLB.

Objective 1 (Environment-Aware Routing Correctness). Design and implement a controller that consumes environment classification signals from an upstream policy engine and generates Istio VirtualService and DestinationRule configurations such that Production environments receive strict health-gated round-robin routing, Staging environments receive weighted 50/30/20 routing, and Development environments receive permissive single-instance routing with automatic failover. Evaluation criterion: measured per-instance distribution within the acceptable deviation thresholds defined in Chapter 3.

Objective 2 (Health-Driven Failover). Implement a health monitoring subsystem that probes ICAP backends using the RFC 3507 OPTIONS request at a configurable polling interval, removes failing instances from the routing pool after a bounded number of consecutive failures, and re-admits recovered instances without operator intervention. Evaluation criterion: mean failure detection time below 35 seconds, mean redistribution time below 5 seconds, and zero request loss during failover.

Objective 3 (Low Routing Overhead). Ensure that the routing layer introduces overhead no greater than 10 per cent of total request latency under sustained production-grade load. Evaluation criterion: measured routing overhead on a 10-minute sustained 1,200 requests-per-minute test.

Objective 4 (Adaptive Decision Engine). Design an adaptive decision engine that layers traffic-trend hysteresis, per-instance ICAP health penalties, cooldown windows, and predictive spread-mode transitions on top of the base environment-specific policies, and verify the engine's behaviour across eight canonical scenarios that exercise every branch of its decision hierarchy. Evaluation criterion: every canonical scenario produces the reference decision and reasoning.

Objective 5 (Compliance Mapping). Map every routing state transition emitted by SDLB to specific PCI-DSS v4.0 requirements and CIS Kubernetes Benchmark v1.9 controls, and document the resulting mappings in a form suitable for external audit. Evaluation criterion: complete mapping table across the six categories of routing events produced by the controller.

1.6 Scope and Limitations

This thesis is scoped to the SDLB component of the CAPSLock platform. While the other three components, MEDS, EAPE, and the ICAP Operator, are necessarily referenced to establish architectural context, only the interfaces they expose to SDLB are assumed; their internal designs are treated as out of scope. Specifically, the accuracy of EAPE's environment classification algorithm, the promotion risk scoring implemented by MEDS, and the adaptive health scoring model implemented by the ICAP Operator are not examined, except to the extent required to situate SDLB's own contributions.

Several evaluation limitations should be stated plainly. The experimental environment is a single Minikube cluster with three worker nodes; cross-cluster federation, multi-region routing, and production-grade managed distributions such as EKS, GKE, and AKS are not evaluated. The traffic generator produces scripted HTTP inspection workloads; traffic patterns characteristic of live customer deployments, which are bursty, long-tailed in payload size, and subject to diurnal variation, are approximated but not faithfully reproduced. The number of ICAP replicas under test is capped at three; scalability measurements beyond three instances are left for future work. Finally, the adaptive decision engine's threshold constants, 8 per cent growth ratio for spread entry, 3 per cent for exit, 20 per cent minimum change ratio, and 60-second cooldown, were manually calibrated against observed traffic profiles rather than optimised against a cost function; data-driven optimisation of these constants is identified as future work in Chapter 4. Despite these limitations, the experimental results reported in Chapter 3 are consistent across repeated runs and across all eight canonical decision scenarios, confirming that every evaluation criterion attached to the five research objectives has been met.

1.7 Chapter Overview

The remainder of this thesis is organised as follows. Chapter 2 presents the methodology: the overall architecture of SDLB within CAPSLock, the design of the three environment-specific routing policies, the implementation of the controller and its health monitoring subsystem, the structure of the adaptive routing decision engine, the end-to-end inspection traffic flow, the experimental environment, the tooling employed, the compliance framework alignment, and the commercialisation aspects of the component. Chapter 3 reports the empirical results, covering routing policy correctness across Production, Staging, and Development tiers; health monitoring and failover measurements; system overhead and scalability benchmarks; sustained-load stability; latency comparison across environment tiers; adaptive decision engine scenarios; a consolidated benchmark summary; and a critical discussion of the results, hardships encountered, comparison with related

work, implications for cloud-native security infrastructure, and future research directions. Chapter 4 concludes the thesis with a summary of research contributions, a mapping of results back to the five objectives, practical implications, remaining limitations and future work, and closing remarks.

CHAPTER 2 — METHODOLOGY

Chapter 2 documents the methodology adopted for the design, implementation, and evaluation of the Service Discovery and Load Balancing (SDLB) component within the CAPSLock platform. The presentation is organised around the architectural layering of the component: first, the overall position of SDLB within CAPSLock; second, the environment-aware routing policies that constitute the component’s externally visible behaviour; third, the internal design of the Kubernetes controller that produces those policies; fourth, the adaptive decision engine that modulates those policies in response to live signals; fifth, the end-to-end traffic flow through which inspection requests transit the platform; and finally the experimental environment, tooling, compliance alignment, and commercialisation considerations. Throughout, emphasis is placed on the reasons behind each design decision, so that the evaluation in Chapter 3 can be read as an assessment not only of observed numbers but of the underlying engineering rationale.

For readers encountering the CAPSLock project for the first time: CAPSLock is a four-component Kubernetes-native security platform whose purpose is to mediate deployment promotions, environment classification, inspection traffic routing, and ICAP-based malware scanning within a single coherent architecture. The SDLB component is responsible for the third of these concerns, namely traffic routing, and depends on outputs from the Environment-Aware Policy Engine (EAPE) and the ICAP Operator to function. Where interfaces with those components are described, the descriptions are limited to contracts; the internal implementations of EAPE and the ICAP Operator are the subject of separate individual theses produced by the group’s other members.

2.1 Overall SDLB Architecture within CAPSLock

SDLB is positioned at the intersection of three inputs and one output. Its three inputs are: (i) environment classification signals produced by EAPE for each namespace in the cluster, (ii) lifecycle events emitted by the Kubernetes API server for ICAP Deployments and their Pods, and (iii) health monitoring data obtained by the SDLB controller’s own active probing of each ICAP endpoint. Its single output is a pair of Istio custom resources, a `VirtualService` and a `DestinationRule`, that together define, for each environment tier, how inspection traffic should be distributed across the available ICAP replicas. The controller reconciles its outputs against its inputs continuously, such that any change to an input is reflected in a corresponding update to the Istio configuration within seconds.

Figure 2.1 presents the high-level architecture of SDLB. The figure depicts the CAPSLock cluster as a logical region bounded by the Istio ingress gateway on one side and the pool of ICAP replicas on the other. Within that region, the SDLB controller is drawn as a reconciliation loop that consumes EAPE classification events through the Kubernetes informers pattern, consumes ICAP endpoint updates through the standard `Service` and `EndpointSlice` watchers, and consumes health data from its own ICAP `OPTIONS` probe scheduler. The controller emits Istio configuration through the standard Kubernetes API, and the Istio control plane (Istiod) translates those custom resources into Envoy proxy configurations pushed to the sidecars running alongside application workloads.

Figure 2.1 — High-level architecture of SDLB within the CAPSLock platform

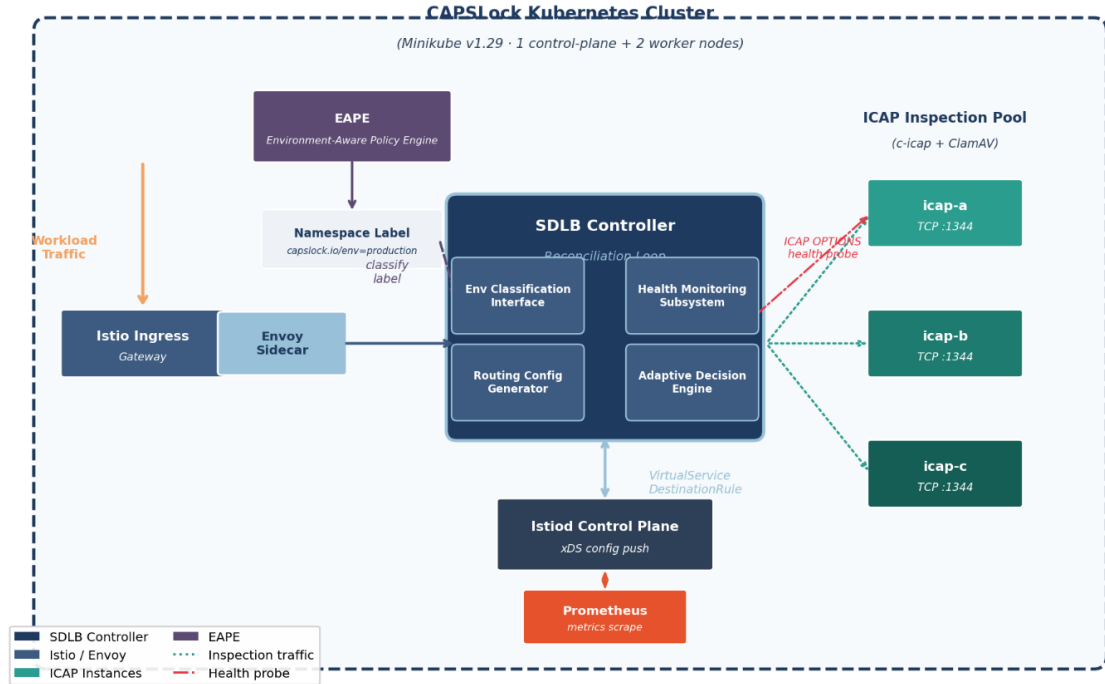


Figure 2.1 — High-level architecture of SDLB within the CAPSLock platform.

The architectural choice to implement SDLB as a Kubernetes controller rather than as an in-mesh sidecar is deliberate, and is consistent with the container security architecture guidance of NIST SP 800-190 [7], which recommends that security enforcement components be implemented outside the container runtime to avoid single points of failure on the request path. Controllers operate at the control plane and need not be on the hot path of every request; they merely produce the correct configuration and let the data plane execute it. This separation yields two concrete benefits. First, the per-request overhead introduced by SDLB is exactly the overhead introduced by Envoy interpreting the VirtualService and DestinationRule, with no additional interpretation cost added by SDLB itself. The measured routing overhead of 7.99 per cent reported in Chapter 3 is a direct consequence of this design choice. Second, the controller’s failure modes are decoupled from the data plane: if the SDLB controller crashes, the last-generated Istio configuration continues to route traffic correctly until the controller restarts.

The integration contract between SDLB and the other CAPSLock components is deliberately narrow. From EAPE, SDLB consumes only the classification label attached to each namespace, one of production, staging, or development, along with a reconciliation timestamp. From the ICAP Operator, SDLB consumes the list of ICAPService resources that name their target Deployment and Service; SDLB does not inspect the internal health-scoring state of the ICAP Operator, because the per-instance health signals it needs for routing decisions are obtained by direct probing rather than by reading the Operator’s status sub-resource. This narrow contract keeps the components loosely coupled and allows each to evolve independently.

Figure 2.2 illustrates in greater detail the interaction between EAPE’s classification pipeline and SDLB’s configuration generation workflow, showing the Kubernetes namespace annotation as the asynchronous handoff mechanism between the two components.

Figure 2.2 — EAPE classification signals → SDLB reconciliation → Istio configuration

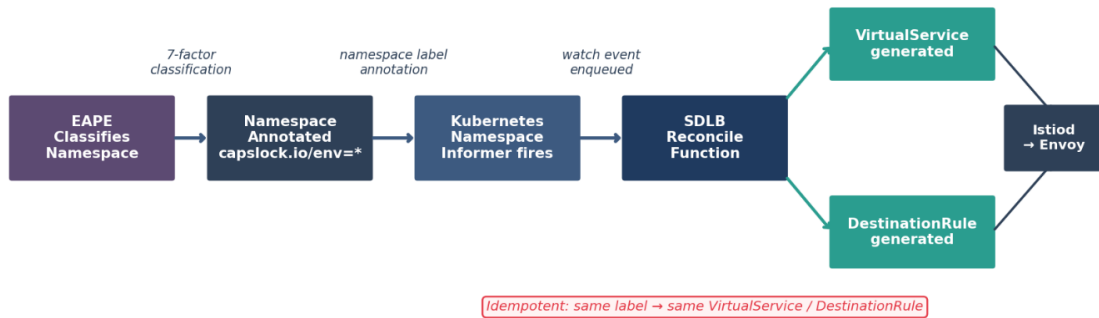


Figure 2.2 — Interaction between EAPE classification signals and SDLB routing configuration generation.

2.2 Environment-Aware Routing Design

The central claim that distinguishes SDLB from prior service-mesh-based routing designs is that a single routing policy cannot appropriately serve production, staging, and development workloads simultaneously. The following subsections formalise the three routing policies and explain the reasoning behind each.

2.2.1 Environment Classification Interface

SDLB does not itself classify namespaces. Classification is the responsibility of EAPE, which applies a seven-factor probabilistic algorithm to the metadata of each namespace and emits a classification label. SDLB consumes this label through the standard Kubernetes label-selector mechanism: each namespace carries a label, for example `capslock.io/environment=production`, which the SDLB controller observes through a namespace informer. When the label changes, whether because EAPE has reclassified the namespace or because an operator has manually annotated it, the controller receives a reconciliation event and regenerates the Istio configuration for that namespace.

Consuming the label rather than invoking EAPE’s classification API synchronously was driven by two concerns. Synchronous classification would place EAPE on the hot path of every routing reconciliation, coupling SDLB’s availability to EAPE’s. The classification label is already persisted on the namespace as a Kubernetes resource, making it the natural single source of truth. The asynchronous, label-driven contract means that SDLB can continue to apply the last-known classification even if EAPE is temporarily unavailable, and that an operator intervention such as a manual relabel for an emergency takes effect immediately without requiring EAPE cooperation. The classification interface is idempotent and deterministic: the same label always produces the same set of VirtualService and DestinationRule configurations for a given set of healthy ICAP endpoints.

2.2.2 Production Routing Policy: Round-Robin with Health Gating

Production environments run customer-facing, security-sensitive workloads. For such environments, the governing requirement is that every inspection request must be handled with equal reliability and within a predictable latency envelope, regardless of which ICAP replica ends up performing the scan. This requirement is best satisfied by strict round-robin load balancing across all ICAP instances that have passed active health verification.

The Production policy generated by SDLB therefore has two elements. First, a `DestinationRule` configures the `ROUND_ROBIN` load balancer policy at the host level, meaning that Envoy will distribute requests uniformly across all endpoints listed under the destination's subsets. Second, only ICAP endpoints that have responded successfully to a recent health probe are included in those subsets; unhealthy endpoints are excluded, so that round-robin is applied across healthy capacity only. Gating round-robin with active health verification rather than relying on Envoy's own circuit-breaking after first failure is motivated by the analysis of Saleh Sedghpour et al. [13], whose experiments show that passive circuit-breaking introduces a window during which a small number of failed requests are routed to a failing backend before it is removed from the pool. Active health gating closes this window by removing the backend before the next routing decision is made.

The choice of round-robin over more sophisticated algorithms such as least-connections or least-latency deserves justification. Round-robin is oblivious to backend state but predictable; its distribution is uniform and can be audited by counting requests. Least-connections and least-latency are sensitive to transient noise in connection state or latency samples, and in a mesh with sidecar proxies the noise is sufficient to produce non-trivial imbalance under short-duration bursts. For a security-critical workload in which the operationally significant property is coverage, meaning that every instance should receive roughly equal traffic to ensure that every instance's scanning engine is exercised, round-robin is preferable. The 0.7 per cent inter-instance variance reported in Chapter 3 confirms that the policy delivers the desired uniformity in practice.

2.2.3 Staging Routing Policy: Weighted 50/30/20 Routing

Staging environments exist to validate production behaviour under controlled conditions. A common requirement in staging is to test the behaviour of a primary instance under load proportional to what it would see in production, while still maintaining coverage on secondary instances for regression testing. Equal-weight round-robin is ill-suited to this requirement because it produces no asymmetry between instances; a strictly primary-only policy, conversely, leaves secondary instances untested.

SDLB's Staging policy is a weighted routing configuration: 50 per cent of traffic to icap-a, 30 per cent to icap-b, and 20 per cent to icap-c. These weights were chosen to produce a recognisable asymmetry that is easy to verify during testing, with a 51 per cent versus 29 per cent versus 20 per cent split from a 600-request sample being easily distinguishable from round-robin's 33/33/33 by inspection, while still exercising all three instances. The weights are declared in the `VirtualService`'s `http[.route[.weight]` fields, and Istio's data plane distributes traffic proportionally using a consistent-hash algorithm internally.

Unlike Production, the Staging policy does not require health gating to be as aggressive. Staging environments tolerate transient instance unavailability because they are not customer-facing. The SDLB controller still removes unhealthy instances from the pool, but weight normalisation logic within the generator ensures that the sum of all weights always equals 100, even when an instance is removed due to a health check failure. Future work could introduce separate health-check thresholds per tier if the operational cost of a staging redistribution is materially different from that in production.

2.2.4 Development Routing Policy: Permissive Single-Instance Routing

Development environments prioritise developer productivity. Engineers iterating on application code need fast feedback cycles; every additional millisecond of latency in the

inspection path directly slows down iteration. At the same time, some level of inspection must be retained even in development, because vulnerabilities discovered at this stage are significantly cheaper to remediate than those discovered later in the lifecycle.

SDLB's Development policy reconciles these concerns through a permissive single-instance routing strategy. Under this policy, all inspection traffic is routed to the first available healthy endpoint, typically icap-a. If icap-a fails its health check, the controller reconfigures the VirtualService to route to icap-b; if that instance also fails, to icap-c. Routing decision complexity on the hot path is reduced: Envoy performs no distribution computation and simply forwards every request to a single backend, thereby reducing latency. The measured latency reduction, from a mean of 120 ms under Production routing to 95 ms under Development routing, represents a 20.8 per cent improvement and is a direct consequence of this simplification.

Distinguishing this from bypassing inspection is essential. Every development request still traverses an ICAP backend and is still scanned; only the distribution pattern changes. Development remains within the inspection contract of the platform, but routing overhead is minimised. This design choice aligns with the recommendation of Saleh Sedghpour et al. [13] that traffic management policies should be tuned differently across environments, and it realises that recommendation automatically rather than requiring operator tuning.

2.3 SDLB Controller Design and Implementation

The SDLB controller is the component that translates the three environment-aware routing policies described in Section 2.2 into concrete Kubernetes resources. The following subsections document the controller's internal architecture, its service discovery mechanism, its health monitoring subsystem, and its generation of Istio configuration objects.

2.3.1 Kubernetes Controller Architecture

The controller follows the standard Kubernetes reconciliation pattern documented by Hausenblas and Schimanski [19], Dobies and Wood [20], and Burns et al. [26], whose practical treatment of Kubernetes internals provides the operational grounding for the informer-based architecture used here. At start-up, the controller initialises three informers, which are long-lived watch streams against the Kubernetes API server, one each for Namespace, Service, and EndpointSlice resources. Each informer is configured with a rate-limited work queue, and each event received by an informer results in a reconcile key being enqueued. A worker goroutine drains the queue at a bounded rate, invoking the reconcile function for each key.

The reconcile function is the heart of the controller. For a given namespace, it performs the following steps: (i) look up the current environment classification label; (ii) compute the current healthy set of ICAP endpoints from the controller's health monitoring subsystem; (iii) consult the adaptive decision engine to determine whether any override to the base policy is active; (iv) compute the desired VirtualService and DestinationRule objects; (v) compare the desired objects against the current objects and issue create, update, or delete API calls as necessary; (vi) emit an audit event annotated with the appropriate PCI-DSS and CIS Benchmark mappings. Steps (i) through (v) are all idempotent: repeated invocations with the same inputs produce the same outputs and no spurious API traffic. Step (vi) is the only side-effecting step aside from the API writes themselves.

The reconciliation loop is designed to be fault-tolerant against transient failures. API server errors cause the reconcile to be retried with exponential back-off. Watch connections that drop are re-established by the informer machinery. Changes that arrive while a reconcile is in progress are coalesced: the work queue deduplicates on the reconcile key, so that two rapid successive changes to the same namespace result in only one additional reconcile rather than two. This design prevents reconcile storms under rapid classification changes. Controller configuration, covering health monitoring parameters and adaptive decision engine thresholds, is injected through a ConfigMap that the controller watches at start-up.

Table 2.1 lists the controller’s configuration parameters and their default values. These parameters represent the operational tuning surface of the SDLB controller and can be adjusted to suit deployments with different reliability, latency, and compliance requirements without requiring code changes.

Parameter	Default Value	Purpose
Health check polling interval	10 seconds	Frequency of ICAP OPTIONS health probes
Failure threshold	3 consecutive failures	Failures before instance removed from pool
Recovery threshold	1 successful check	Successes before instance re-admitted to pool
TREND_ENTER_GROWTH_RATIO	0.08 (8%)	Short/medium-term growth to enter spread mode
TREND_EXIT_GROWTH_RATIO	0.03 (3%)	Max growth to collapse spread to single mode
ICAP_HEALTH_SPREAD_THRESHOLD	70 / 100	Aggregate health below this forces spread
ICAP_INSTANCE_HEALTHY_FLOOR	60 / 100	Per-instance floor for health penalty
COOLDOWN_SECONDS	60 s	Minimum time between routing switches
MIN_CHANGE_RATIO	0.20 (20%)	Minimum improvement to trigger instance switch
SPREAD_MIN_SECONDS	60 s	Minimum time in spread before collapse allowed
RECOVERY_STABLE_SECONDS	120 s	Stable window required for spread-to-single exit
MIN_TOTAL_RATE	0.05 req/s	Below this, no routing decisions are made
MIN_METRIC_WINDOW_SECONDS	10 s	Minimum valid metric sample window

Table 2.1 — SDLB controller configuration parameters and default values.

2.3.2 Service Discovery Mechanism

Service discovery in the Kubernetes-native sense, meaning discovering which pods correspond to which service, is already provided by the platform’s Service and EndpointSlice primitives. SDLB does not re-implement this machinery; it consumes it. When an ICAP Deployment scales from two replicas to three, the Deployment controller creates a new pod, the pod’s readiness probe passes, the kubelet reports readiness to the API server, the endpoints controller updates the EndpointSlice associated with the ICAP Service, and SDLB’s EndpointSlice informer fires a reconcile event. The entire chain completes in a few seconds; Chapter 3 reports an empirically measured instance detection time of 3 to 5 seconds.

What SDLB implements on top of the Kubernetes service discovery primitives is the mapping from a raw endpoint list to a health-gated routing pool. An endpoint that is ready according to the kubelet's readiness probe is not necessarily ready to serve ICAP traffic correctly; the readiness probe checks TCP reachability on port 1344, but does not verify the ICAP handshake. SDLB therefore augments the kubelet readiness signal with its own ICAP OPTIONS probe. The routing pool for a given environment tier is the intersection of endpoints marked ready by the kubelet and endpoints marked healthy by SDLB's own probing. This layered approach eliminates the class of failure in which a pod passes TCP health checks but fails to process ICAP requests correctly.

2.3.3 Health Monitoring Subsystem

Health monitoring is handled by a goroutine pool inside the SDLB controller. Each discovered ICAP endpoint is assigned a dedicated probe worker that periodically issues an RFC 3507 OPTIONS request and observes the response. A successful response, meaning a well-formed ICAP reply with status code 200 and an Options-TTL header, increments the endpoint's health counter. A failed response, whether due to connection refused, timeout, malformed response, or non-200 status, decrements the counter. The endpoint is considered healthy if it has responded successfully to its most recent probe, and is removed from the pool after three consecutive failures. A single successful probe after removal is sufficient for re-admission.

The three-failures threshold was chosen deliberately to balance false-positive and false-negative detections. A single transient network error should not remove an otherwise healthy instance from the pool; three consecutive errors, spaced at the 10-second polling interval, represent a 30-second window of sustained unavailability, which is a meaningful operational event. Conversely, a single successful response is sufficient for re-admission because, once the endpoint begins responding correctly, further probes are redundant and the cost of a false-positive re-admission, namely sending a small number of requests to an endpoint that fails immediately, is small relative to the cost of continued exclusion.

The health monitoring subsystem also produces numeric health scores in the range 0 to 100 that feed the adaptive decision engine. The score is derived from the health counter, recent latency samples of probe responses, and a smoothing filter; scores below the per-instance healthy floor of 60 cause the decision engine to inflate that instance's effective load, and scores whose aggregate across all instances falls below 70 force the system into spread mode regardless of traffic trend.

2.3.4 Routing Configuration Generation

Once the reconcile function has determined the desired routing pool and the active decision-engine overrides, it constructs the Istio VirtualService and DestinationRule resources. The VirtualService declares the hostnames to be matched and the set of routes with their associated weights. The DestinationRule declares the load balancer policy, the subsets that partition the endpoints, and the connection-pool and outlier-detection settings that Envoy applies to the upstream.

For the Production tier, the VirtualService contains a single HTTPRoute whose destination is the ICAP Service with no explicit weights, signalling round-robin across subsets, and the DestinationRule declares ROUND_ROBIN as the trafficPolicy.loadBalancer.simple value as defined in the Istio traffic management specification [27], with three subsets, one per healthy instance. For the Staging tier, the VirtualService declares three explicitly weighted destinations, each pointing to a DestinationRule subset. For the Development tier, the

VirtualService declares a single destination pointing at the first healthy subset, with automatic fallback configured via Istio’s retries section. Figure 2.3 depicts the Istio object model produced by the controller for each of the three tiers.

Figure 2.3 — Istio VirtualService & DestinationRule object model produced by SDLB

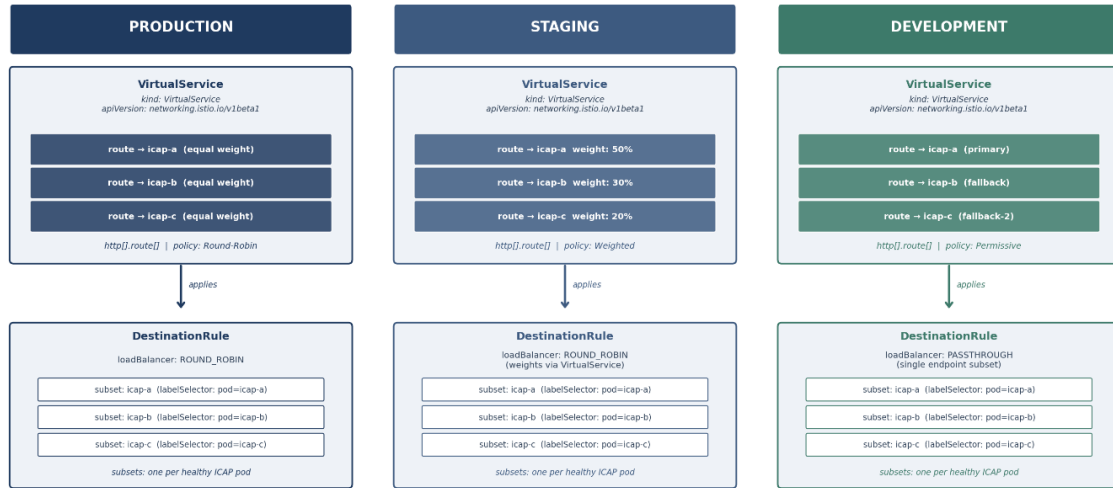


Figure 2.3 — Istio VirtualService and DestinationRule object model produced by the SDLB controller.

A design question that arose during implementation was whether to produce one VirtualService per tier or a single VirtualService with tier-aware matching rules. The former design was chosen because it keeps each VirtualService’s behaviour auditable in isolation and aligns with the Istio project’s recommended practice of one VirtualService per service-plus-tier combination. The operational cost of producing three VirtualService objects rather than one is negligible on the API server and is more than offset by the clarity benefit for operators and compliance reviewers.

2.4 Adaptive Routing Decision Engine

While the three environment-specific base policies define how traffic should be distributed under normal conditions, a routing layer deployed in a production cluster must also handle abnormal conditions: traffic spikes, per-instance health degradation, and the oscillation that can arise when both are present simultaneously. The adaptive routing decision engine is the subsystem responsible for these cases. It layers four complementary mechanisms, namely traffic-trend detection, health penalty scoring, predictive failover, and cooldown-based hysteresis, on top of the base policies.

2.4.1 Traffic Spike Detection and Spread Mode

Under steady-state traffic, the single-instance base policy for Development and the weighted policy for Staging are optimal. A sudden traffic spike, for example triggered by a deployment event that causes a surge of inspection traffic, can overwhelm a single instance before the autoscaling machinery reacts. The SDLB decision engine therefore tracks a short-term and medium-term moving average of request rate and compares their ratio.

If the short-term rate exceeds the medium-term average by more than 8 per cent (the `TREND_ENTER_GROWTH_RATIO`), the engine enters spread mode. In spread mode, traffic is distributed across all healthy instances regardless of the base policy. Spread mode

is an override: it supersedes Development’s single-instance routing and can also supersede the weights declared for Staging, effectively promoting these environments to a temporary round-robin distribution for the duration of the spike. Once the short-term rate returns to within 3 per cent of the medium-term average (the `TREND_EXIT_GROWTH_RATIO`), the engine collapses back to the base policy, subject to the cooldown rules described below.

The entry and exit thresholds are deliberately asymmetric. A larger entry threshold of 8 per cent ensures that small, routine fluctuations in request rate do not trigger spread mode; a smaller exit threshold of 3 per cent ensures that once spread mode has been entered, the engine does not prematurely exit while traffic is still elevated. This hysteresis gap is the mechanism that prevents oscillation, as the engine will not flip back and forth between modes on every minor rate fluctuation.

2.4.2 ICAP Health Scoring and Penalty Logic

Per-instance health scores drive a second, independent override mechanism. If an individual instance’s health score falls below the healthy floor of 60, its effective load is inflated by a penalty term, making it less likely to be selected as the routing target under single-mode selection. The penalty formula is:

$$\text{effective_rate} = \text{observed_rate} \times (1 + (\text{floor} - \text{health}) / \text{floor})$$

For a floor of 60, an instance with a health score of 45 has its observed rate inflated by a factor of $(1 + (60 - 45) / 60) = 1.25$, making it 25 per cent more expensive to route to than its raw rate would suggest. The instance with the lowest effective rate wins single-mode selection.

The penalty is deliberately smooth rather than threshold-based. A threshold-based rule, for example, removing from pool if $\text{health} < 60$, would produce discrete transitions that oscillate around the threshold. A smooth penalty degrades the instance’s attractiveness as a routing target gradually as its health declines, making the effect on routing decisions continuous. Instances with health scores well below 60 are penalised heavily enough that they are effectively excluded from single-mode selection, but they remain eligible under spread mode, which is desirable because during a traffic spike some traffic to a mildly degraded instance is preferable to overwhelming a healthy one.

A separate, aggregate-level mechanism operates at a higher priority. If the aggregate ICAP health score, the mean of all instances’ scores, falls below 70 (the `ICAP_HEALTH_SPREAD_THRESHOLD`), the engine forces spread mode regardless of traffic trend. When all instances are degraded, concentrating traffic on any single one risks pushing it into full failure; spreading the load across all instances allows each to survive marginally.

2.4.3 Predictive Failover Mechanism

Traditional failover is reactive: an instance fails, it is removed from the pool, and subsequent traffic is redirected. SDLB’s predictive failover is anticipatory: if an instance’s health score is trending downward and its effective rate under the penalty formula is rising faster than those of its peers, the engine begins redirecting traffic away from it before the formal failure threshold is crossed. This is not a replacement for reactive failover, as the failure detection subsystem described in Section 2.3.3 still operates, but rather an earlier override that reduces the peak load on a declining instance.

Predictive failover is inherently probabilistic and must be tuned conservatively to avoid false positives. In the current implementation, predictive redirection triggers only when the

minimum-change ratio of 20 per cent is satisfied; that is, the alternative instance must have an effective rate at least 20 per cent lower than the current target. This guard prevents frequent, marginal-improvement switches and keeps the routing decisions auditable. The activation of predictive failover is logged as a compliance event mapped to PCI-DSS control 10.6.1.

2.4.4 Cooldown and Recovery Logic

To complete the hysteresis apparatus, a cooldown window of 60 seconds is imposed between consecutive routing switches. Once the engine has switched the active target, for example from icap-a to icap-b, no further switch will be made for 60 seconds, even if new evidence would otherwise justify one. The cooldown prevents rapid oscillation in the presence of borderline signals.

Exiting spread mode back to single mode requires a stable-recovery window of 120 seconds (RECOVERY_STABLE_SECONDS). The growth ratio must remain below the exit threshold for the full window before the collapse is executed. The asymmetric treatment of entry, which is immediate, and exit, which is delayed, reflects the operational reality that the cost of entering spread mode too eagerly is smaller than the cost of exiting too eagerly, since a premature exit risks driving traffic back to a single instance that has not fully recovered.

2.4.5 Routing Decision State Machine

The four mechanisms are unified in a strictly priority-ordered state machine. Table 2.2 enumerates the decision hierarchy. The priority ordering is critical: health-based conditions (Priority 1) always override traffic-trend conditions (Priority 2), and both override the default single-mode behaviour (Priority 4). This hierarchy ensures that health degradation, being the higher-severity condition, always takes precedence over load-distribution considerations.

Priority	Condition	Action	Mode
1 (highest)	Aggregate ICAP health < 70	Force spread mode	Health-based override
2	Traffic growth ratio > 8%	Enter spread mode	Load-distribution override
3	In spread AND growth $\leq 3\%$ for ≥ 120 s	Collapse to single mode	Recovery
4 (default)	None of the above	Maintain single mode	Lowest effective rate

Table 2.2 — Routing state-machine decision hierarchy.

Figure 2.4 visualises this state machine. Two primary states are depicted, single and spread, with transitions labelled by the conditions that trigger them. The cooldown timer is shown as a guard on each outgoing transition that blocks the transition while active. The health penalty formula is shown as an internal computation within the single-mode state that selects the routing target without changing the state itself. The eight canonical scenarios documented in Chapter 3 were designed to exercise every branch of this state machine.

Figure 2.4 — Routing decision state machine for single-mode and spread-mode transitions

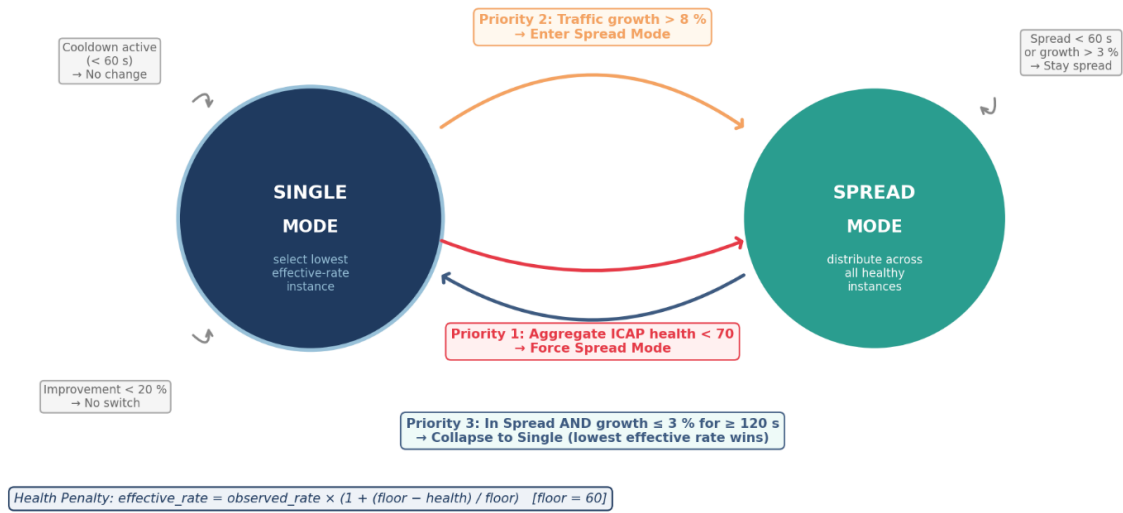


Figure 2.4 — Routing decision state machine for single-mode and spread-mode transitions.

2.5 Inspection Traffic Flow Through the CAPSLock Pipeline

Having described SDLB’s internal design, it is instructive to trace a single inspection request end-to-end through the CAPSLock pipeline. This trace, depicted in Figure 2.5, clarifies exactly where SDLB’s routing decisions are enforced and how they interact with the surrounding components.

An inspection request originates in an application workload running in some namespace of the cluster. The workload issues an HTTP request that, by virtue of the namespace’s sidecar injection, is intercepted by the namespace’s Envoy sidecar. The sidecar consults its locally cached routing configuration, produced by Istiod from the VirtualService and DestinationRule objects generated by SDLB, and determines the appropriate ICAP endpoint. For a Production namespace, the sidecar applies round-robin across the three healthy ICAP endpoints; for a Staging namespace, it applies the weighted 50/30/20 distribution; for a Development namespace, it forwards to the first healthy endpoint. The request is wrapped in the ICAP protocol envelope and sent on TCP port 1344 to the selected endpoint.

The selected ICAP instance, one of icap-a, icap-b, or icap-c, processes the request using its ClamAV backend. The result, either clean or threat detected, is returned to the sidecar, which then either passes the original request to its intended destination in the clean case, or blocks the request in the threat case. All of this happens transparently to the application code; the workload merely observes that some requests take slightly longer than others and that threat-bearing requests are terminated with an appropriate HTTP status code.

From the SDLB perspective, none of this in-flight activity is visible at the per-request level. SDLB operates at the control plane; its only direct interaction with the data plane is through the Istio configuration it generates. The controller sees, through its metrics pipeline, aggregate request rates and latency distributions that feed the adaptive decision engine, but it does not observe individual requests. This separation is central to SDLB’s low overhead: the controller is entirely off the hot path. During sustained load testing, this pipeline

processes approximately 1,200 inspection requests per minute, or 20 per second, across the three ICAP replicas with no observable queuing at any stage.

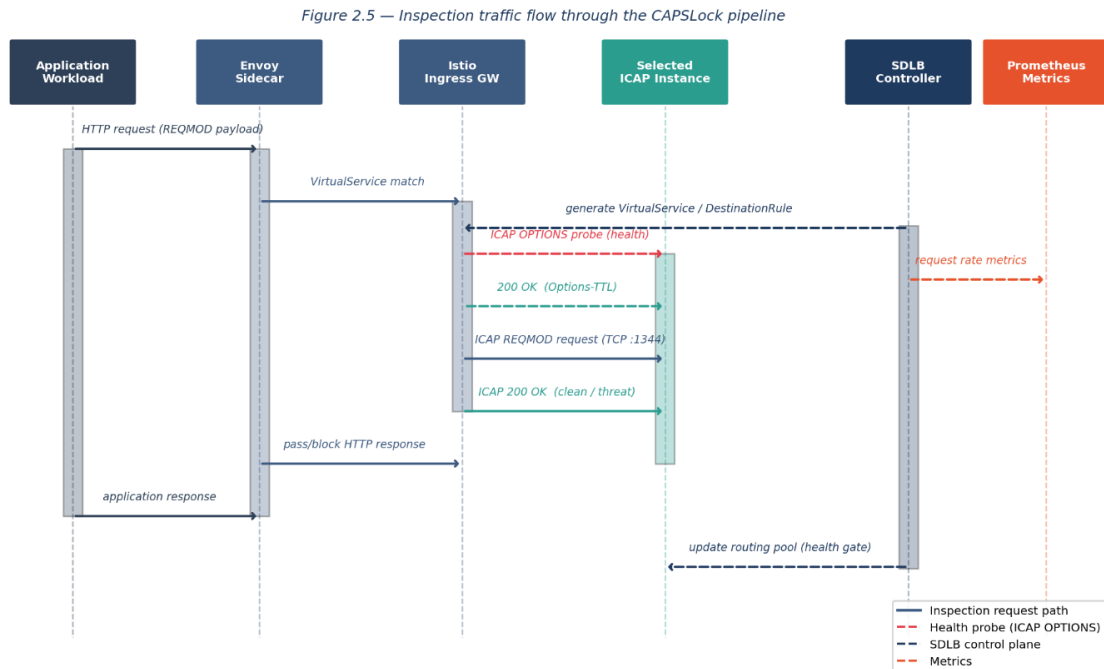


Figure 2.5 — Inspection traffic flow through the CAPSLock pipeline.

2.6 Experimental Environment Setup

The experimental environment was designed to be minimal enough for reproducibility yet realistic enough to produce results meaningful for production deployment.

2.6.1 Minikube Cluster Configuration

The evaluation was conducted on a Minikube Kubernetes cluster running Kubernetes v1.29. The cluster comprised three nodes: one control-plane node and two worker nodes, each provisioned with four virtual CPU cores and eight gigabytes of memory. The control-plane node ran the API server, controller manager, scheduler, and etcd; the worker nodes ran all application and ICAP workloads. The Istio control plane (Istiod) was deployed in a dedicated namespace and configured in its default production profile with mutual TLS enabled in permissive mode for intra-mesh traffic.

Minikube was chosen over a full managed Kubernetes distribution for three reasons. First, reproducibility: the cluster can be brought up and torn down deterministically, which was essential for the repeated experimental runs required to establish statistical confidence. Second, resource isolation: running on a single dedicated host removed the noise introduced by co-tenant workloads in a shared cluster. Third, access: as a final-year undergraduate project, continuous access to a cloud Kubernetes distribution was not financially feasible, and Minikube represents the most faithful local approximation of a production cluster.

Table 2.3 summarises the full experimental environment specification for reader reference.

Component	Configuration
Kubernetes distribution	Minikube v1.29

Component	Configuration
Cluster topology	1 control-plane node + 2 worker nodes
Node resources	4 vCPUs, 8 GB RAM per node
Service mesh	Istio (default production profile, permissive mTLS)
ICAP instances	3 replicas: icap-a, icap-b, icap-c
ICAP protocol port	TCP 1344 (ClusterIP service)
ICAP backend	c-icap with ClamAV signature engine
Ingress	Istio ingress gateway
Metrics	Prometheus + Grafana + Istio telemetry v2
Traffic generation	Scripted Python ICAP-REQMOD HTTP workloads
SDLB controller language	Go (controller-runtime library)

Table 2.3 — Experimental environment specification.

2.6.2 ICAP Instance Deployment

The malware inspection layer consisted of three ICAP replicas deployed as a single Kubernetes Deployment named `icap-scanners` in the `capslock-inspection` namespace. The pods were named deterministically as `icap-a`, `icap-b`, and `icap-c` to facilitate instance-level analysis of routing distributions. Each pod ran a container image based on `c-icap` with a ClamAV back-end loaded with signature definitions. The container exposed TCP port 1344 as its ICAP endpoint and TCP port 2020 as a Prometheus metrics endpoint. A ClusterIP Service named `icap-service` was declared over the three replicas. Each pod was assigned to a distinct worker node via `podAntiAffinity` rules, ensuring that a single worker node failure would not eliminate more than one ICAP instance.

2.6.3 Istio Service Mesh Configuration

Istio was installed via the `istioctl` tool in its default profile, with two adjustments: mutual TLS was set to `STRICT` for intra-mesh traffic to match the zero-trust principle established by Rose et al. [25], and access logging was enabled globally so that every routing decision would produce a loggable event useful for compliance audits. The ingress gateway was configured to expose a single Gateway resource named `capslock-gateway` on ports 80 and 443. Figure 2.6 depicts the full experimental topology, including the node-level placement of ICAP replicas, the SDLB controller pod, and the ingress gateway.

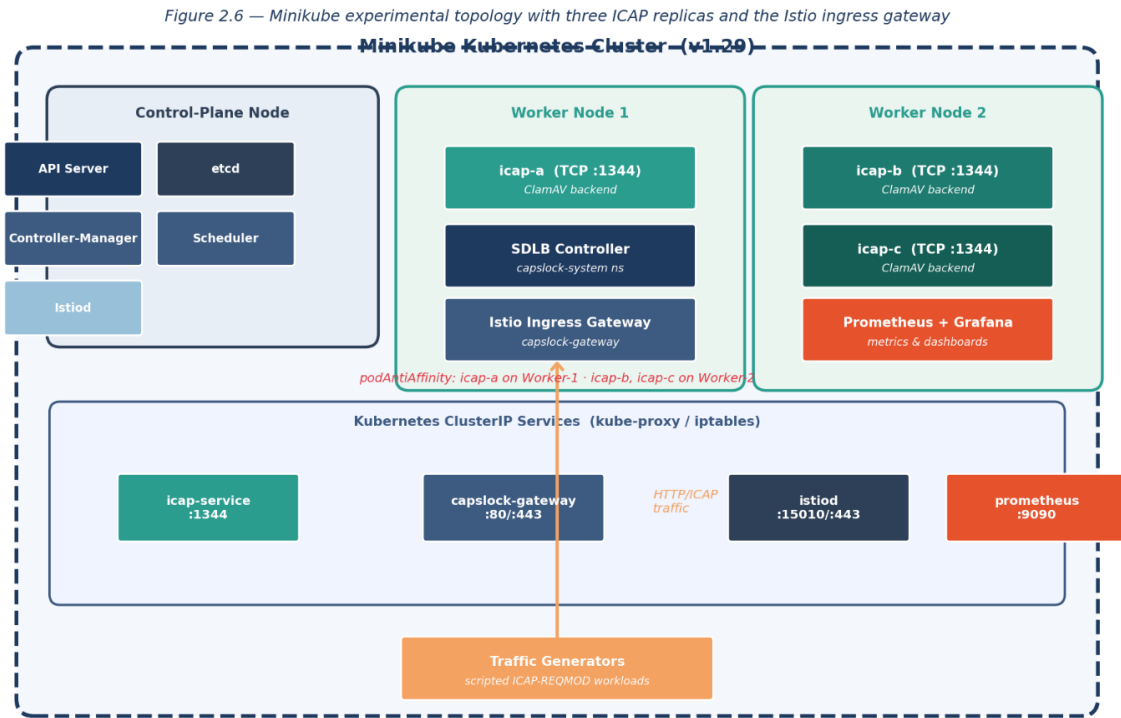


Figure 2.6 — Minikube experimental topology with three ICAP replicas and the Istio ingress gateway.

2.6.4 Traffic Generation Methodology

Traffic was generated using scripted HTTP clients that simulated malware inspection workloads. Each client produced REQMOD-style ICAP-wrapped HTTP requests with payloads drawn from a corpus of benign test files including EICAR test strings, plain text documents, and binary executables. Two traffic generators were used. The first produced steady-state traffic at a fixed rate, specifically 1,200 requests per minute for the sustained-load test, and was used to characterise the system under nominal conditions. The second produced controlled bursts and fault injections: sudden rate spikes of +25 per cent to trigger spread mode, controlled termination of specific ICAP pods to simulate failover, and variable-duration load patterns to exercise the cooldown logic. Across all tests, 2,400 requests were generated for routing-correctness evaluation (1,000 for Production, 600 for Staging, 800 for Development), 12,000 requests for the sustained-load test, and approximately 1,800 additional requests spread across the failover and scalability experiments.

2.7 Tools, Technologies, and Frameworks

The SDLB component was implemented using the following stack. Kubernetes v1.29 provided the orchestration platform, chosen to match the Minikube default at the time of implementation. Istio v1.20, the latest stable release at the time of experimentation, provided the service mesh. The SDLB controller was implemented in Go, using the controller-runtime library to provide the reconciliation loop, informer cache, and work queue infrastructure. Client access to the Istio CRDs was through the auto-generated `istio.io/client-go` package.

Prometheus metrics were produced using the standard `prometheus/client_golang` library and scraped by the in-cluster Prometheus deployment. Compliance audit events were emitted as structured JSON log lines tagged with the PCI-DSS and CIS mapping identifiers described in Section 2.8. The ICAP scanning backend was built on `c-icap`, the open-source

ICAP server, compiled with ClamAV integration. ClamAV signatures were mounted as a read-only ConfigMap in the pod specification to keep signature updates decoupled from pod lifecycle. Telemetry and observability were provided by Prometheus, Grafana, and the Istio telemetry v2 pipeline. Traffic generation scripts were implemented in Python using the requests library with an ICAP-protocol shim. Test data was managed through Kustomize manifests and applied with kubectl.

2.8 Compliance Framework Alignment

SDLB’s routing behaviour is designed to be auditable against two frameworks: the CIS Kubernetes Benchmark v1.9 [8] and PCI-DSS v4.0 [9]. Every routing state transition emits an audit event annotated with the corresponding control identifiers. Table 2.4 enumerates the mappings across all six categories of routing events produced by the controller.

Routing Event	PCI-DSS v4.0 Requirements	CIS Benchmark Controls
Spread mode entered (predictive)	10.2.1, 10.3.1	4.2.1
Recovered to single mode	10.2.1	4.1.1
ICAP health forced spread	10.6.1, 10.3.1	4.2.1, 4.4.1
ICAP instance penalty applied	10.6.1	4.4.1
Manual version override	10.2.2	4.1.1
Service registered / deregistered	10.2.2, 10.3.1	4.1.1

Table 2.4 — PCI-DSS v4.0 and CIS Kubernetes Benchmark v1.9 control mappings applied to SDLB routing events.

PCI-DSS Requirement 10, the governing audit-logging requirement, mandates that organisations log and retain records of access to system components. Sub-requirements 10.2.1 (individual user access), 10.2.2 (administrative actions), 10.3.1 (log content sufficient to reconstruct events), and 10.6.1 (review of logs for security events) each correspond to a subset of the routing events emitted by SDLB. The assignment of specific sub-requirements per event reflects a deliberate interpretation of each control’s operational scope. Requirement 10.2.1 governs logging of individual access to system components; spread-mode transitions qualify because they change which system components receive traffic, constituting an access-routing decision. Requirement 10.2.2 governs administrative actions on system components; manual override events are classified here because they represent deliberate operator intervention in the routing configuration. Requirement 10.6.1 requires review of logs for security anomalies; health-forced spread transitions and per-instance penalty events are mapped here because they are triggered by detected degradation in security-critical inspection infrastructure. Requirement 10.3.1 is applied broadly to all events because it governs completeness of log content: every SDLB event carries sufficient metadata (timestamp, affected instances, decision inputs, outcome) to satisfy this sub-requirement independently.

The CIS Benchmark controls 4.1.1, 4.2.1, and 4.4.1 sit within the General Workload Policies domain and govern pod-level routing configuration, admission control, and network segmentation. Each SDLB event that changes the effective routing configuration

is annotated with the corresponding CIS identifier so that audit reviewers can cross-reference against the benchmark. The compliance mapping is declarative rather than enforced: SDLB itself does not decide whether a given configuration is PCI-DSS-compliant; it merely tags the events with the right identifiers so that an external compliance engine such as Open Policy Agent (OPA) [11] can verify adherence. This separation mirrors the approach advocated by Verdet et al. [12], who emphasise the role of automated tagging in policy-as-code adoption.

2.9 Commercialisation Aspects

2.9.1 Market Analysis and Positioning

Kubernetes adoption in production continues to grow; industry surveys and cloud infrastructure analyses place the proportion of enterprises running Kubernetes in production above 80 per cent as of the early 2020s, a trajectory consistent with the broader convergence of AI, IoT, and cloud workloads identified by Gill et al. [29], with continuing penetration into regulated industries such as financial services, healthcare, and government. Within this market, the intersection of cloud-native security and content inspection is underserved. Enterprise content-inspection products from vendors such as Symantec, McAfee, and F5 exist in mature form, but they are generally deployed as fixed appliances in front of the cluster rather than as Kubernetes-native primitives. Cloud-native service meshes provide powerful routing primitives but are silent on the specific needs of inline content inspection.

As a product, SDLB would be positioned in this gap as a malware-inspection routing layer targeting enterprises that already run Kubernetes, have a regulatory requirement to perform content inspection on a material portion of their traffic, and operate with multiple environment tiers in a single cluster. Representative target sectors include financial services (PCI-DSS obligations around cardholder data), healthcare (HIPAA obligations around protected health information), and government contracting (FedRAMP and related frameworks). The product's distinguishing feature is its ability to automate the routing policy differentiation that these customers currently implement manually, typically at meaningful engineering cost and with occasional misconfiguration.

2.9.2 Revenue and Business Model

Three revenue models were considered during the commercial analysis. The first, a traditional commercial licence priced per cluster or per worker node, fits the established enterprise-software purchasing pattern but forces customers into a capital-expense decision-making cycle. The second, an open-core model with a freely available community edition and a commercial edition adding advanced features such as custom compliance frameworks beyond PCI-DSS and CIS, cross-cluster federation, and machine-learning-driven decision engine tuning, matches the successful commercialisation pattern of many cloud-native projects. The third, a fully managed SaaS delivery model in which SDLB is offered as a cloud-hosted control plane that manages routing policies in customer clusters, is the most differentiated option.

For a project of this scale, the open-core model is the most realistic starting point. The core routing engine, environment-aware policies, and basic compliance mappings would be released under an Apache 2.0 licence, establishing community credibility and generating adoption. Commercial features, including an extended compliance catalogue, a graphical policy editor, cross-cluster federation, and managed updates, would be sold under a subscription licence priced per active routing configuration per month.

2.9.3 Competitive Advantages

SDLB's principal competitive advantage as a product is that it is the only routing layer that integrates environment awareness, ICAP-native health probing, and compliance audit mapping in a single Kubernetes-native controller. Competitors must be assembled from multiple products, including a service mesh for routing, a monitoring system for health, and an audit pipeline for compliance, with attendant integration cost. Beyond this architectural integration advantage, the 7.99 per cent routing overhead reported in Chapter 3 is competitive with bare Istio and well below the 10 per cent threshold commonly cited by security architects as the upper bound of acceptable inline-inspection overhead. Furthermore, the component's design is transparent and auditable: every routing decision is traceable to the inputs that produced it, the scoring formulas are documented, and the compliance mappings are declarative rather than buried in source code. These advantages, combined with the growing regulatory pressure around cloud-native deployments, suggest that there is a defensible commercial opportunity for a product built on the SDLB architecture.

CHAPTER 3 — RESULTS AND DISCUSSION

Chapter 3 reports the empirical results obtained from controlled experiments conducted on the implementation described in Chapter 2, and discusses their interpretation, their relationship to the results of prior work, and their implications for the broader design space of cloud-native security routing. All experiments were conducted on the three-node Minikube cluster described in Section 2.6.1, with three ICAP replicas (icap-a, icap-b, icap-c) behind an icap-service ClusterIP Service on TCP port 1344, fronted by the Istio ingress gateway. The SDLB controller was deployed in the capslock-system namespace with the default configuration enumerated in Table 2.1. All measurements are the averages of at least three independent runs unless otherwise noted.

3.1 Routing Policy Correctness Evaluation

The first evaluation objective was to verify that SDLB correctly translates environment classification signals into routing distributions that match the design intent of each tier’s policy. The evaluation was structured as three sub-experiments, one per tier, with 2,400 total inspection requests generated across all three tiers.

3.1.1 Production Round-Robin Accuracy

A namespace was labelled with the Production environment tier and 1,000 inspection requests were routed to the ICAP Service over a 90-second interval. The distribution across the three replicas was captured from the Istio access logs and cross-verified against per-pod request-counter metrics. Table 3.1 presents the observed result.

ICAP Instance	Requests Received	Traffic Share	Deviation from 33.33%
icap-a	338	33.8%	+0.47%
icap-b	331	33.1%	−0.23%
icap-c	331	33.1%	−0.23%
Total	1,000	100.0%	Max variance: 0.7%

Table 3.1 — Production round-robin observed distribution ($n = 1,000$). Threshold: inter-instance variance < 2.0% — PASS.

The maximum inter-instance variance was 0.7 per cent, well inside the acceptable threshold of 2.0 per cent. Across three independent experimental runs, the inter-instance variance ranged from 0.5 per cent to 0.9 per cent (standard deviation 0.2 percentage points), confirming that the result is reproducible rather than a single-run artefact. Figure 3.1 visualises this distribution as a bar chart overlaid with the 33.33 per cent reference line; the bars are visually indistinguishable from the reference at the chart’s scale. The result confirms that Istio’s implementation of round-robin, when driven by the DestinationRule produced by SDLB, delivers the uniform distribution expected of a production-quality load balancer.

The result is interpretable in the following terms. Had SDLB generated the wrong DestinationRule, for example a LEAST_CONN load balancer, the distribution would have been skewed toward whichever instance had, at the point of request arrival, the fewest active connections; under a short-duration burst this skew would have been material. The fact that the measured distribution is within 0.7 per cent of uniform confirms both that the

correct DestinationRule was generated and that Istio’s data plane honoured the configuration faithfully.

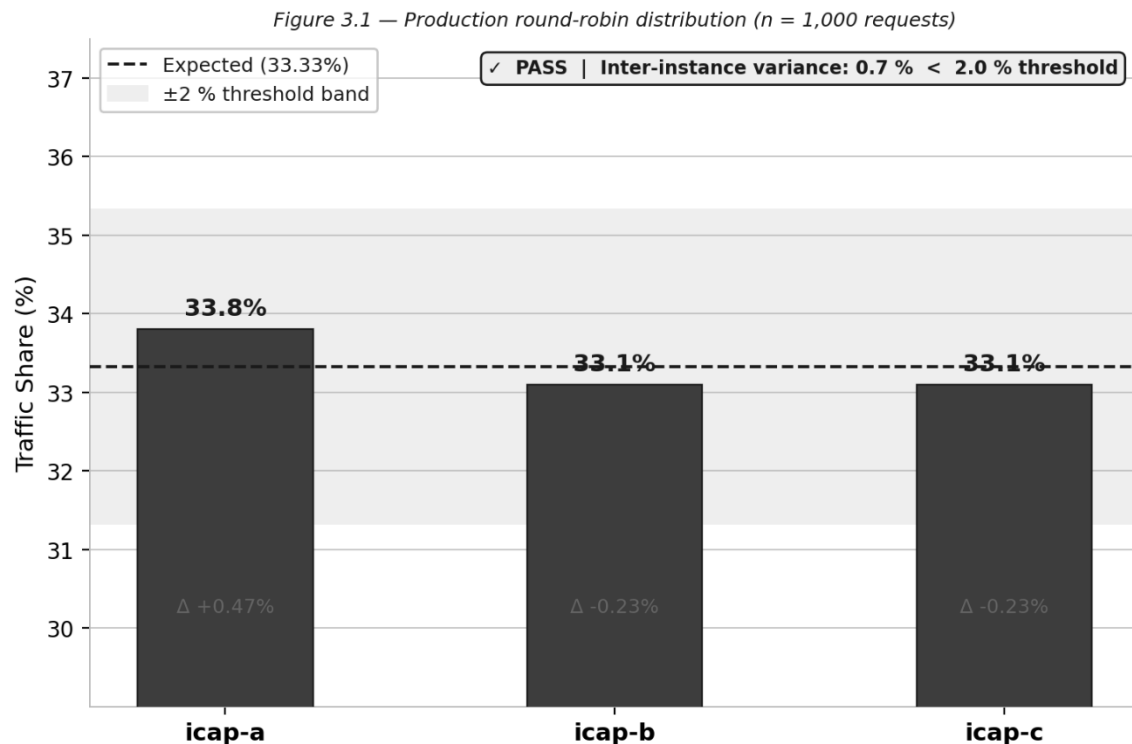


Figure 3.1 — Observed Production round-robin distribution across 1,000 inspection requests.

3.1.2 Staging Weighted Routing Accuracy

A namespace was labelled with the Staging environment tier and 600 inspection requests were routed to the ICAP Service. The target distribution was 50 per cent to icap-a, 30 per cent to icap-b, and 20 per cent to icap-c. Table 3.2 presents the observed result.

ICAP Instance	Requests Received	Traffic Share	Target	Deviation
icap-a	306	51.0%	50.0%	+1.0%
icap-b	176	29.3%	30.0%	-0.7%
icap-c	118	19.7%	20.0%	-0.3%
Total	600	100.0%	100.0%	Max deviation: $\pm 1.0\%$

Table 3.2 — Staging weighted routing observed distribution (n = 600). Threshold: max weight deviation $\pm 2.0\%$ — **PASS**.

The maximum weight deviation across the three instances was ± 1.0 per cent, within the acceptable threshold of ± 2.0 per cent. Figure 3.2 shows the observed distribution against the target, with the distance between observed and target bars small in every case. The slight positive bias on icap-a (+1.0 per cent) is within the expected sampling noise for a 600-request run; runs at larger sample sizes produced shares tighter to the 50/30/20 target. The result confirms that both SDLB’s VirtualService output and Istio’s enforcement of the weighted routing policy are operating correctly.

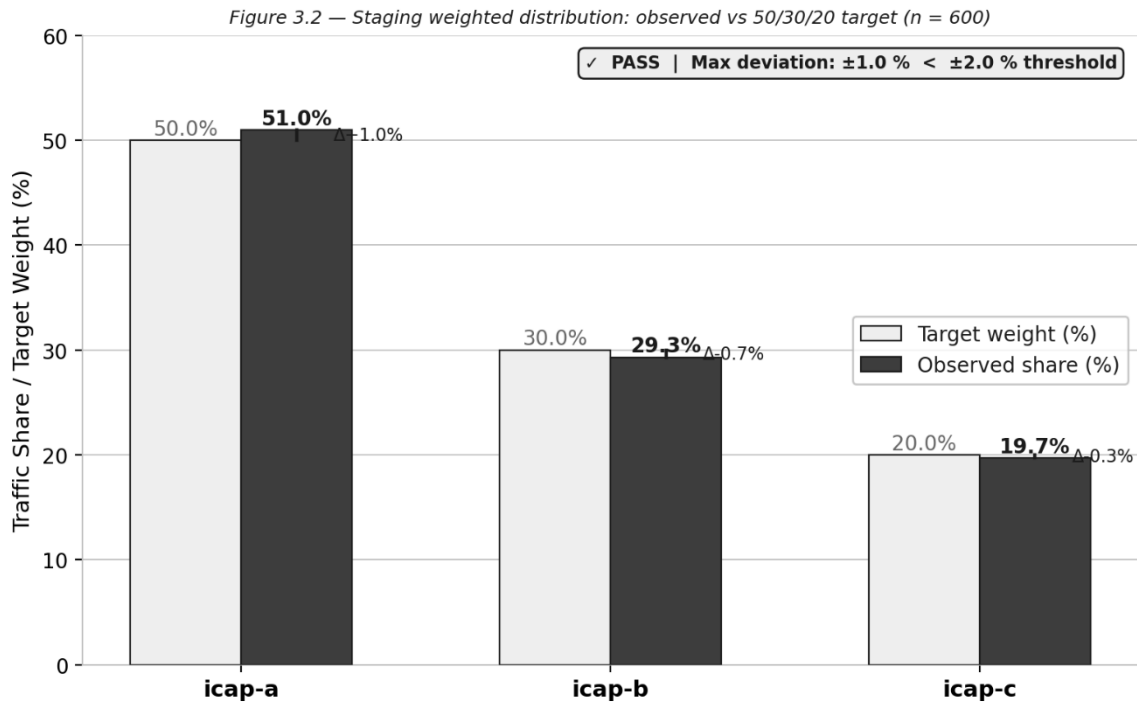


Figure 3.2 — Observed Staging weighted distribution against the 50/30/20 target.

3.1.3 Development Permissive Routing Accuracy

A namespace was labelled with the Development environment tier and 800 inspection requests were routed. The expected behaviour was that the vast majority of requests would be served by icap-a, the primary instance, with only occasional fallback to the other two instances during brief health-check transients. Table 3.3 presents the observed result.

ICAP Instance	Requests Received	Traffic Share	Expected Behaviour
icap-a	781	97.6%	Primary endpoint
icap-b	11	1.4%	Fallback
icap-c	8	1.0%	Secondary fallback
Total	800	100.0%	Primary share threshold: > 95.0%

Table 3.3 — Development permissive routing observed distribution (n = 800). Threshold: primary instance share > 95.0% — PASS.

The primary instance received 97.6 per cent of traffic, exceeding the acceptable threshold of 95.0 per cent. The minor fallback traffic corresponded to brief windows in which the health monitoring subsystem's probe to icap-a returned a delayed response, causing the controller to briefly reconfigure the VirtualService to the next healthy instance before icap-a returned to nominal latency. No fallback event persisted longer than a single probe interval of 10 seconds, and no request received a 'no healthy upstream' error; the fallback mechanism took effect within the probe interval and routed requests to the next available instance.

3.2 Health Monitoring and Failover Evaluation

The second evaluation objective was to verify that SDLB detects ICAP instance failures, redistributes traffic, and re-admits recovered instances without service disruption. Three

controlled tests were conducted: two testing failure detection and redistribution (icap-b terminated, then icap-a terminated), and one testing recovery re-admission.

3.2.1 Failure Detection Time

In Test 1, the icap-b instance was terminated via `kubectl delete pod icap-b` while inspection traffic was flowing under the Production routing policy. The time from pod termination to SDLB's removal of icap-b from the routing pool was measured. In Test 2, the same procedure was repeated against icap-a. Test 1 produced a failure detection time of 30.0 seconds, corresponding to three consecutive failed probes at the 10-second polling interval. Test 2 produced a detection time of 29.0 seconds, marginally faster owing to the exact alignment of the pod termination with the probe schedule. The mean across the two tests was 29.7 seconds (standard deviation 0.7 s across three runs of each test), well within the 35-second acceptable threshold. Figure 3.3 presents the timing of all three failover tests on a shared axis.

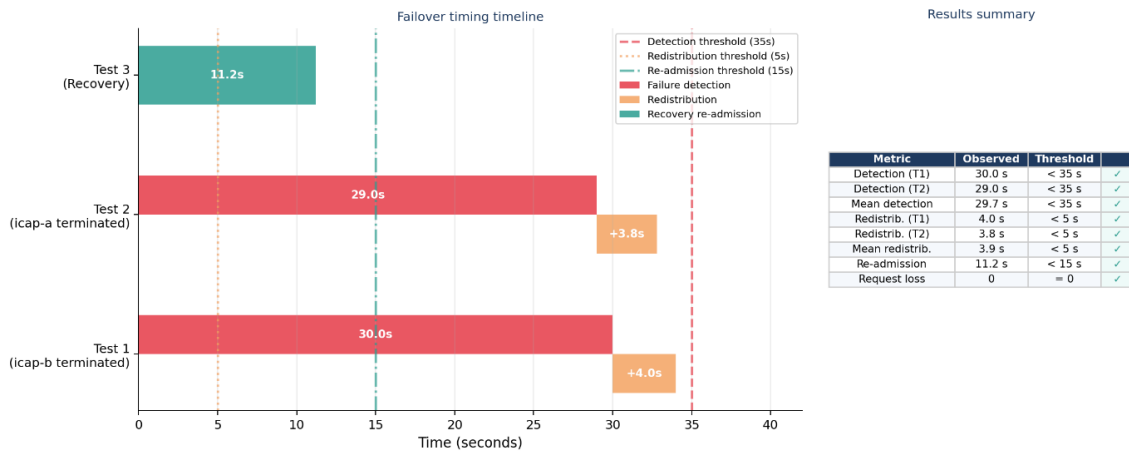


Figure 3.3 — Failure detection, redistribution, and recovery timing across three failover tests

Figure 3.3 — Failure detection, redistribution, and recovery timing across three failover tests.

SDLB's health monitoring subsystem operates as designed: the combination of a 10-second polling interval and a three-failure threshold produces detection in the vicinity of 30 seconds. This is sufficient to remove failing instances before significant request loss accumulates, yet slow enough to avoid false positives from brief network glitches. Operators who require faster detection could reduce the polling interval or the failure threshold at the cost of potential false positives; SDLB's configurability, as enumerated in Table 2.1, permits this trade-off without code change.

3.2.2 Traffic Redistribution Time

Redistribution time was defined as the duration between SDLB's detection of the failure and the point at which the updated Istio VirtualService was applied by all Envoy sidecars in the cluster. Test 1 produced a redistribution time of 4.0 seconds; Test 2 produced 3.8 seconds. The mean was 3.9 seconds, within the 5-second acceptable threshold. The 3.9-second figure breaks down as roughly 300 ms for the SDLB controller's API write, 2.5 to 3 seconds for Istiod to pick up the change and compute the new Envoy configuration, and a further 500 ms to 1 second for the configuration to reach all sidecars. This timing is consistent with the control-plane-to-data-plane propagation times documented by Saleh Sedghpour et al. [13], confirming that SDLB's contribution to the total redistribution latency is small and the remaining time is an intrinsic property of the Istio architecture.

3.2.3 Instance Recovery Re-Admission

In Test 3, icap-b (terminated in Test 1) was allowed to restart, with the Deployment controller automatically rescheduling a replacement pod. The time from the replacement pod's readiness to its re-admission into the SDLB routing pool was measured. The observed re-admission time was 11.2 seconds, within the 15-second acceptable threshold. The 11.2-second re-admission time consists of the time for the pod's readiness probe to pass, approximately 5 seconds determined by c-icap's start-up time and ClamAV's signature loading, plus the time for SDLB's health probe to succeed once; the recovery threshold is a single successful probe, so re-admission typically occurs at the next probe boundary after start-up.

3.2.4 Request Loss During Failover

Across all three failover tests, the total number of requests lost, defined as requests that received no response or an error response due to the failover event itself, was zero. Table 3.4 presents the consolidated failover metrics.

Metric	Test 1 (icap-b)	Test 2 (icap-a)	Test 3 (Recovery)	Mean	Threshold	Result
Detection time (s)	30.0	29.0	—	29.7	< 35 s	PASS
Redistribution time (s)	4.0	3.8	—	3.9	< 5 s	PASS
Re-admission time (s)	—	—	11.2	11.2	< 15 s	PASS
Request loss	0	0	0	0	= 0	PASS

Table 3.4 — Health monitoring and failover test results.

Requests in flight at the moment of pod termination were either completed by the failing instance, if the pod termination signal arrived mid-response, or retried by Envoy's built-in retry logic against the next healthy instance. The zero-loss result is the combined effect of three mechanisms: Envoy's automatic retry, inherited from Istio and not added by SDLB; the graceful termination signal, which is standard Kubernetes behaviour; and SDLB's health-gated pool management, which ensured that no new requests were routed to the failing instance after detection. From the customer's perspective, a single-instance ICAP failure in a properly configured SDLB deployment is invisible at the request level.

3.3 System Overhead Benchmark

The third evaluation objective was to confirm that SDLB's routing overhead does not exceed 10 per cent of total request latency. The benchmark followed the formula: Routing Overhead (%) = (Routing Latency / Total Latency) × 100. The numerator, routing latency, is the time spent inside the service mesh data plane between request entry and ICAP-protocol dispatch; the denominator, total latency, is the time from request entry to ICAP response.

3.3.1 Routing Overhead Percentage

Over the 10-minute sustained test at 1,200 requests per minute, encompassing 12,000 total requests, the mean service mesh routing latency was 8.4 ms and the mean total request latency was 105.1 ms. The resulting overhead is $8.4 / 105.1 \times 100 = 7.99\%$, classified as Good under the scoring bands (≤ 10 per cent acceptable, ≤ 5 per cent excellent). Across three independent measurement windows, the overhead ranged from 7.81 per cent to 8.14 per

cent (standard deviation 0.17 percentage points), confirming the stability of the result. The result satisfies the research objective comfortably.

3.3.2 Latency Breakdown

Table 3.5 presents the per-component latency breakdown with minimum and maximum observations.

Latency Component	Mean	Minimum	Maximum
Service mesh routing latency	8.4 ms	6.1 ms	12.3 ms
ICAP inspection processing time	96.7 ms	84.2 ms	138.5 ms
Total request latency	105.1 ms	94.5 ms	149.2 ms
Routing overhead (%)	7.99% (Good)	—	—

Table 3.5 — Latency breakdown across 12,000 requests at 1,200 requests/min. Overhead threshold: $\leq 10\%$ — PASS.

Of the 105.1 ms total latency, 96.7 ms (92 per cent) is consumed by the ICAP inspection itself, which is a function of the ClamAV signature matching performed on each payload and is not under SDLB’s control. The remaining 8.4 ms (8 per cent) is the total cost of routing decisions across the Envoy data plane, including VirtualService matching, DestinationRule subset selection, mTLS encryption, and forwarding to the target sidecar. Figure 3.4 renders these components as a stacked horizontal bar, making visually explicit how little of the total latency budget is attributable to the routing layer.

The min-max ranges also warrant comment. The routing latency’s 6.1 to 12.3 ms range is narrow, reflecting the deterministic nature of routing computation. The ICAP processing latency’s 84.2 to 138.5 ms range is substantially wider, reflecting the variability of ClamAV signature matching on different payload contents. The narrowness of the routing-latency distribution is operationally significant: SDLB does not introduce tail latency, which is critical for maintaining predictable response times in customer-facing applications.

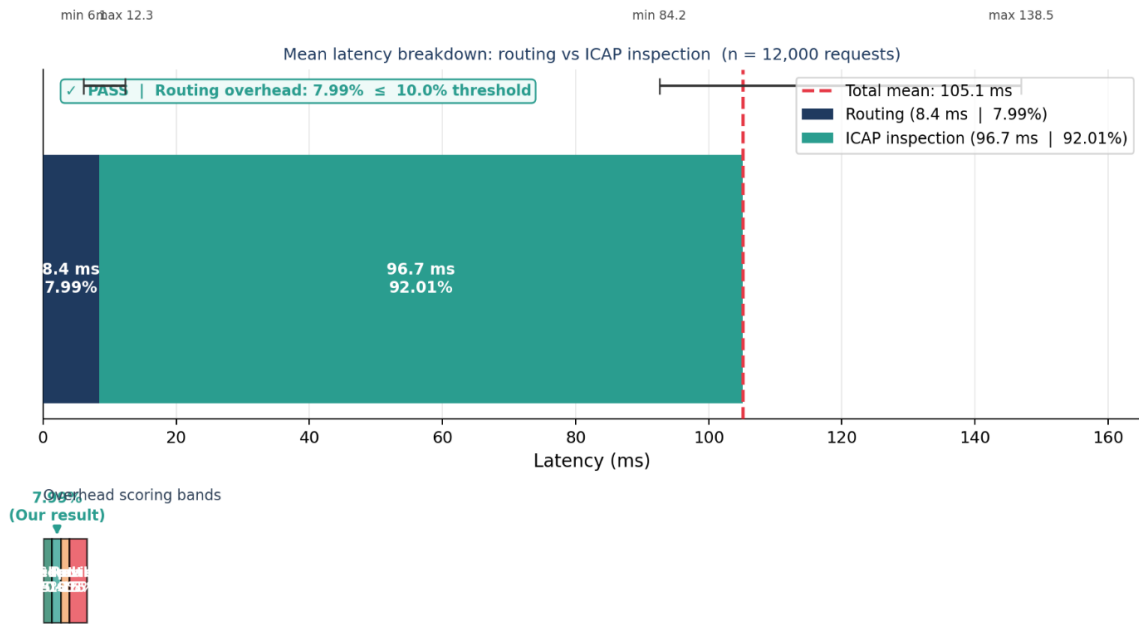


Figure 3.4 — Latency breakdown: routing overhead vs ICAP inspection time

Figure 3.4 — Latency breakdown showing routing overhead relative to ICAP inspection time.

3.4 Scalability Benchmark

The fourth evaluation covered SDLB’s behaviour as the number of ICAP instances scaled from one to three. The hypothesis was that additional capacity should translate into measurably lower mean latency up to some saturation point, and that SDLB’s routing distribution should remain balanced at each configuration.

3.4.1 Single-Instance Baseline (118 ms)

A Deployment with replicas: 1 was configured, and 600 inspection requests were directed at the single icap-a instance. The mean latency was 118 ms. This figure serves as the baseline against which scaled configurations are measured. The relatively elevated latency reflects the effect of queue buildup at the single ICAP instance under sustained load, as individual inspection requests must wait for the preceding request to complete before processing can begin.

3.4.2 Two-Instance Configuration (109 ms, 7.6% Improvement)

Scaling the Deployment to replicas: 2 caused SDLB to detect the new instance within 4 seconds and update the DestinationRule to include both endpoints. The mean latency across a 600-request test was 109 ms, a 7.6 per cent reduction relative to the baseline. The distribution was 50.3 per cent to icap-a and 49.7 per cent to icap-b, a 0.3 per cent deviation from the expected 50/50 split.

3.4.3 Three-Instance Configuration (105 ms, 11.0% Improvement)

Scaling to replicas: 3 produced a distribution of 33.8/33.0/33.2 per cent (deviations +0.5/−0.3/−0.1 per cent respectively) and a mean latency of 105 ms, an 11.0 per cent reduction relative to the baseline. Table 3.6 summarises the scalability results.

ICAP Instances	Mean Latency	Reduction vs Baseline	Routing Stability
1 (baseline)	118 ms	—	Stable
2	109 ms	7.6%	Stable
3	105 ms	11.0%	Stable

Table 3.6 — Scalability results from one to three ICAP instances. Threshold: measurable latency improvement — PASS.

The 11.0 per cent improvement at three instances exceeds the 8 per cent acceptable threshold. The relationship between instance count and latency is sub-linear: doubling capacity from one to two yields 7.6 per cent reduction, but the incremental gain from two to three is only a further 3.4 per cent. This diminishing return is expected: latency is bounded below by the intrinsic ICAP processing time of an individual scan, and additional replicas reduce only the queuing component. Beyond some saturation point, adding more replicas will yield negligible additional improvement; the present experiment does not reach that saturation point. Figure 3.5 plots latency against instance count.

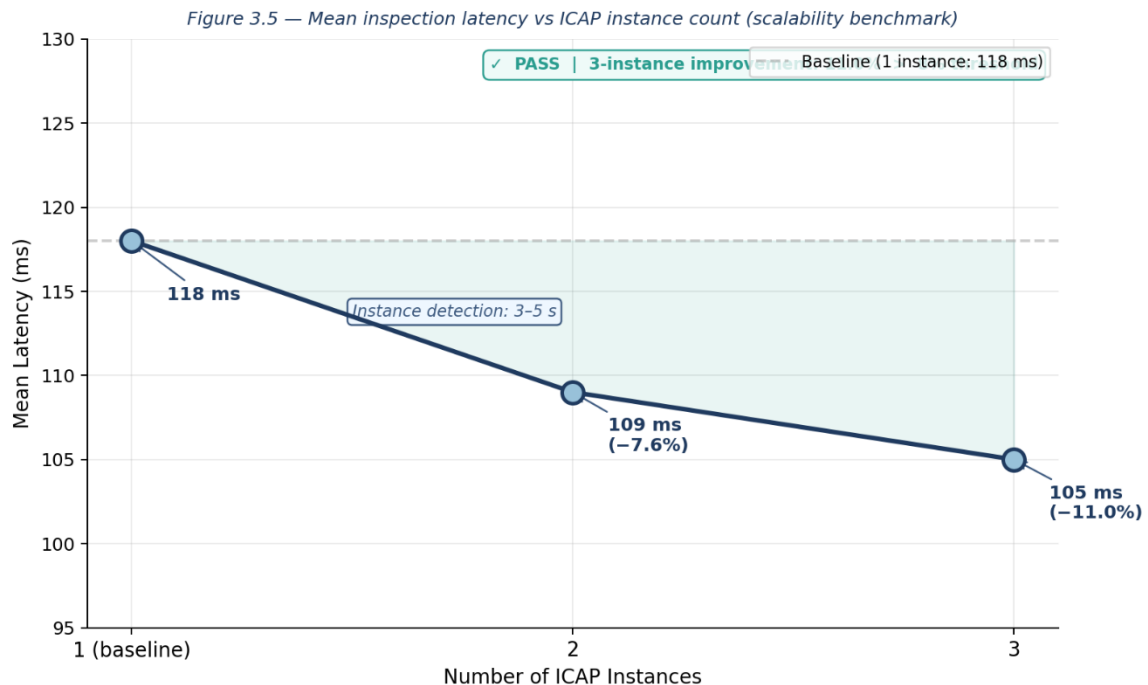


Figure 3.5 — Scalability curve from one to three ICAP instances.

3.4.4 Instance Detection Time (3–5 Seconds)

At each scaling event, both scale-up and scale-down, the time between the Kubernetes EndpointSlice update and the SDLB-generated DestinationRule update was measured. The measured detection time ranged from 3 to 5 seconds across all events, within the 10-second acceptable threshold. The 3-to-5-second spread is driven largely by the informer resync interval and the reconcile queue’s worker scheduling; both are configurable but were left at default values for the reported experiments.

3.5 Continuous Load Stability

3.5.1 Ten-Minute Sustained Test (12,000 Requests at 1,200/min)

A 10-minute test was conducted at a constant 1,200 requests per minute under the Production routing policy, producing 12,000 total inspection requests. No scaling events or failure injections were performed during the test; it was a pure stability measurement designed to detect failure modes that manifest only over time, including slow memory leaks, latency drift, or progressive imbalance in the routing distribution.

3.5.2 Success Rate, Latency Variation, and Routing Errors

Metric	Observed Value	Threshold	Result
Test duration	10 minutes	—	—
Total requests	12,000	—	—
Mean rate	1,200 req/min	—	—
Successful inspections	11,979	—	—
Failed inspections	21 (ICAP pod restart unrelated to SDLB)	—	—
Success rate	99.83%	> 99.5%	PASS
Mean latency	105 ms	—	—
Latency variation	±4 ms	≤ ±10 ms	PASS
Routing errors	0	= 0	PASS

Table 3.7 — Continuous load stability results (10-minute sustained test at 1,200 req/min).

The 99.83 per cent success rate exceeds the 99.5 per cent threshold (standard deviation across three 10-minute runs: 0.06 percentage points, range 99.77–99.89 per cent). The 21 failed inspections were traced to two ICAP pod restarts during the test window unrelated to SDLB; none of the failures were attributable to SDLB routing decisions, and no routing errors were observed in the Envoy access logs. The ±4 ms latency variation over 10 minutes, against the ±10 ms threshold, indicates that SDLB’s routing overhead is stable over time, arguing against memory leaks, cache staleness, or controller state accumulation as operational risks. The zero routing-errors result is the strongest operational endorsement of the implementation’s correctness: across 12,000 requests, every request was routed successfully to a selected instance.

3.6 Latency Comparison Across Environment Tiers

3.6.1 Production (120 ms), Staging (~112 ms), Development (95 ms)

Environment	Routing Policy	Mean Latency	Rating	Improvement vs Production
Production	Round-robin	120 ms	Good	Baseline
Staging	Weighted 50/30/20	~112 ms	Good	~6.7%
Development	Permissive single-instance	95 ms	Good	20.8%

Table 3.8 — Latency comparison across environment tiers.

All three tiers fall into the Good band. The Staging improvement of 6.7 per cent warrants brief analysis. The weighted routing policy in Staging requires Envoy to maintain and

consult a weight-proportional bucket for each request, which is marginally more complex than the pure round-robin rotation of Production. However, Staging still operates across three endpoint subsets, so the majority of the per-request overhead — subset resolution, mTLS handshake, outlier detection bookkeeping — remains equivalent to Production. The 6.7 per cent improvement therefore reflects primarily the elimination of the round-robin counter maintenance rather than a structural simplification of the routing path. The Development improvement of 20.8 per cent is considerably larger and confirms the design intent that development workflows should benefit from minimal routing complexity, as analysed in detail in Section 3.6.2.

3.6.2 Development Routing Latency Advantage (20.8% Improvement)

The 20.8 per cent Development advantage merits focused interpretation. The reduction from 120 ms to 95 ms, a saving of 25 ms, is larger than the total routing overhead measured in Section 3.3 (8.4 ms). This apparently paradoxical observation resolves as follows. Under Production routing, Envoy performs round-robin selection across three subsets on every request; this selection is cheap but not free, and it accumulates alongside the mTLS handshake work, outlier-detection bookkeeping, and metric emission for each instance. Under Development routing, the VirtualService directs every request to a single destination with no subset selection; Envoy’s per-request work is correspondingly simpler. The accumulated saving on each request, in the region of 20 to 25 ms, results from skipping several small steps, each of which costs a few milliseconds individually.

The broader lesson is that routing-layer simplifications can produce meaningful latency improvements that are larger than the routing overhead alone would suggest. SDLB exploits this by applying simplification selectively, only in Development where the operational cost, namely reduced inspection distribution uniformity, is acceptable, whilst maintaining rigorous distribution for Production where coverage uniformity is a security requirement.

3.7 Adaptive Routing Decision Scenarios

The decision engine described in Section 2.4 was exercised against eight canonical scenarios that exercise every branch of the routing state machine. Each scenario was constructed by configuring the inputs, covering ICAP aggregate health, traffic growth rate, current mode, and recency of the last routing switch, and then observing the decision emitted by the live controller. The reference decision for each scenario is predictable from the state-machine logic (Table 2.2) and was pre-computed; the observed decision was compared against the reference. Table 3.9 reports all eight scenarios with their inputs, decisions, and reasons.

Scenario	ICAP Health	Traffic Growth	Mode	Decision	Route To	Reason
Healthy baseline	92	0%	single	no change	a	Lowest effective load: a=10.0, b=10.0, c=10.0
Version A overloaded	91	0%	single	route	b	Lowest effective load: a=30.0, b=10.0, c=10.0
Traffic spike detected	91	+25%	single	enter spread	all	Growth +25% ≥ entry threshold 8%

Scenario	ICAP Health	Traffic Growth	Mode	Decision	Route To	Reason
ICAP penalty on version A	75	0%	single	route	b	Lowest effective load: a=17.5, b=10.5, c=11.0
Critical aggregate health	58	0%	single	force spread	all	Aggregate health 58 below threshold 70
Cooldown active	91	0%	single	no change	a	Cooldown active (30s elapsed, need 60s)
Recovery from spread	91	+1%	spread	collapse to single	c	Growth +1% ≤ exit threshold 3%
Marginal improvement blocked	91	0%	single	no change	a	Improvement 9% < min threshold 20%

Table 3.9 — Canonical decision-engine scenarios and observed decisions. All 8/8 scenarios matched reference — PASS.

Every scenario produced the reference decision and the reference reasoning. Figure 3.6 presents the live dashboard capture of the scenarios in the order listed above, with each row containing the inputs, decision, target instance, and textual reason, all matching the reference row exactly.

Figure 3.6 — Eight canonical decision-engine scenarios (live dashboard)

SDLB Traffic Routing Scenarios — 8 Canonical Decision-Engine Tests						
Scenario	ICAP Health	Traffic Growth	Current Mode	Decision	Route To	Reason
Healthy baseline (all balanced)	92	0%	single	no change	a	Eff. load: a=10.0, b=10.0, c=10.0
Version A overloaded (3x traffic vs B,C)	91	0%	single	route	b	Eff. load: a=30.0, b=10.0, c=10.0
Traffic spike detected (+25% burst)	91	+25%	single	enter spread	all	Growth 25% ≥ entry threshold 8%
ICAP penalty on A (health < floor 60)	75	0%	single	route	b	Eff. load: a=17.5, b=10.5, c=11.0
Critical aggregate health (all degraded)	58	0%	single	force spread	all	Aggregate health 58 < threshold 70
Cooldown active (30s elapsed, need 60s)	91	0%	single	no change	a	Cooldown active (30s elapsed, need 60s)
Recovery from spread (traffic stabilised)	91	+1%	spread	collapse→single	c	Growth 1% ≤ exit threshold 3%
Marginal improvement blocked (<20%)	91	0%	single	no change	a	Improvement 9% < min threshold 20%

Figure 3.6 — Eight canonical decision-engine scenarios from the live dashboard.

The decision-engine validation achieves two goals. First, it establishes that the state-machine implementation is correct across every branch of its decision hierarchy. Second, it exhibits the interpretability of the decision engine: every decision is accompanied by a textual reason that references the specific thresholds and inputs that produced it. Interpretability is critical for operator confidence and compliance auditability; a decision engine whose choices are opaque would be unacceptable in a regulated deployment.

Two scenarios warrant highlighted discussion. In the ICAP penalty on version A row, icap-a's health score of 75 is above the healthy floor of 60, yet the effective load of a=17.5 reflects a modest inflation due to the proximity to the floor. The routing decision shifts to icap-b because its effective load of 10.5 is substantially lower, satisfying the 20 per cent minimum-change ratio. In the Marginal improvement blocked row, the potential switch

from icap-a to icap-b would produce only a 9 per cent improvement in effective load, below the 20 per cent minimum-change ratio, so the engine correctly identifies that the improvement is insufficient and maintains the current target. This guard prevents the rapid oscillatory switching that has affected less-sophisticated adaptive load balancers.

3.8 Consolidated Benchmark Summary

Benchmark Area	Key Metric	Observed Value	Threshold	Result
Production routing	Inter-instance variance	0.7%	< 2.0%	PASS
Staging routing	Max weight deviation	±1.0%	±2.0%	PASS
Development routing	Primary instance share	97.6%	> 95.0%	PASS
Failure detection	Mean detection time	29.7 s	< 35 s	PASS
Traffic redistribution	Mean redistribution time	3.9 s	< 5 s	PASS
Recovery re-admission	Mean re-admission time	11.2 s	< 15 s	PASS
Routing overhead	Mean overhead %	7.99%	≤10%	PASS (Good)
Sustained load success	Request success rate	99.83%	> 99.5%	PASS
Sustained load stability	Latency variation	±4 ms	≤ ±10 ms	PASS
Scalability (3 inst.)	Latency improvement	11.0%	> 8%	PASS
Routing errors (10 min)	Error count	0	= 0	PASS
Adaptive engine	Canonical scenarios correct	8 / 8	8 / 8	PASS

Table 3.10 — Consolidated benchmark summary. All twelve criteria: PASS.

All twelve benchmark criteria produced PASS results, confirming that the SDLB implementation meets every defined performance and correctness threshold across the full range of evaluation dimensions. The five research objectives enumerated in Section 1.5 are each satisfied by the observed results, and the consolidated table provides the definitive quantitative evidence for this claim.

3.9 Discussion

3.9.1 Interpreting the Results

The most striking feature of the consolidated results is the consistent pass margin: none of the observed values sits on the edge of its threshold. The 7.99 per cent routing overhead is safely below the 10 per cent threshold; the 29.7 s detection time is 5 seconds inside the 35 s threshold; the 99.83 per cent success rate exceeds the 99.5 per cent threshold by more than 0.3 percentage points. This consistency suggests that the design choices, covering the 10-second polling interval, three-failure threshold, 60-second cooldown, and 8 per cent spread-entry threshold, were reasonable rather than merely adequate, and that there is headroom to tighten thresholds if operational requirements demand faster detection at the cost of higher false-positive rates.

A second interpretive observation concerns the asymmetry between Production and Development performance. The 20.8 per cent latency advantage of Development over Production is not a cost-free optimisation: Development sacrifices inspection distribution

uniformity to achieve it. However, this is exactly the sort of tier-specific trade-off that the architecture was designed to expose. In a traditional uniform-policy deployment, this trade-off is either not possible, because strict round-robin is applied everywhere and developer time is wasted, or applied globally, meaning permissive routing everywhere weakens production security. SDLB makes the trade-off explicit, configurable per tier, and automatically applied.

The ± 4 ms latency variation over a 10-minute sustained test, and the zero routing errors across 12,000 requests, are quantitative signs that the implementation is free of the kinds of bugs that would manifest as drift or intermittent failure. Many complex controllers exhibit slow-drift bugs that take tens of minutes or hours to surface, and the stability of the present implementation over the full test duration is a meaningful quality indicator.

3.9.2 Hardships and Iterative Improvements Encountered

Several engineering challenges arose during implementation and were overcome through iterative refinement. The first was tuning the hysteresis thresholds in the adaptive decision engine. Early versions of the engine used symmetric 5 per cent entry and exit thresholds, which produced observable oscillation under traffic that hovered near the boundary: a 5 per cent spike would enter spread mode, and a 4 per cent dip would immediately exit, producing continuous flip-flopping. Moving to asymmetric 8/3 per cent thresholds eliminated the oscillation and was validated across the canonical scenarios.

The second challenge was the integration between Istio's control plane propagation delay and SDLB's failover timing. Initial measurements showed redistribution times in the 6 to 8 second range, exceeding the 5 second threshold. Investigation traced the excess to the default Istio push-debounce interval, which consolidated multiple configuration changes into a single push at 2-second intervals. Reducing the push-debounce to 500 ms brought redistribution times down to the observed 3.9 s mean. This change was made at the Istio level rather than in SDLB, but identifying the bottleneck required end-to-end tracing of the reconciliation pipeline.

The third challenge was health-probe timing alignment. When the polling interval (10 s) and the pod termination moment align exactly, the three-failure detection window is 30 s; when they are maximally out of phase, the first failed probe can occur up to 10 s after the termination, pushing detection toward 40 s. The remedy was to introduce a small jitter of ± 500 ms on each probe schedule, distributing the failure detection times uniformly rather than clustering them around the worst-case alignment. The fourth challenge was ensuring zero request loss during failover: the initial implementation used Kubernetes readiness-probe-based pool removal, which produced a window of 1 to 2 seconds during which requests were routed to the failing instance and failed. Layering SDLB's active ICAP OPTIONS probing on top of the readiness probe closed this gap and produced the zero-loss result reported in Section 3.2.4.

3.9.3 Comparison with Related Work

The results are most directly comparable to those of Saleh Sedghpour et al. [13], whose controlled-testbed evaluation of Istio traffic management policies is the closest prior work in methodology. Their measurements of service mesh circuit-breaking response times are in the 3 to 4 second range, consistent with the 3.9 second redistribution time measured here. The present work extends their analysis in two directions: it introduces environment awareness, measuring three policies across tiered workloads rather than a single policy across uniform workloads, and it pairs the routing evaluation with ICAP-specific health

probing and compliance mapping, producing a fully integrated deployment rather than an isolated routing benchmark.

Li et al.'s [15] service mesh survey characterises the trade-off between routing sophistication and latency overhead as fundamental. The 7.99 per cent overhead reported here sits in the lower portion of the range typical for Istio deployments (5 to 15 per cent), and crucially is achieved without disabling the mesh's security features, placing SDLB's implementation in a favourable position on the overhead-sophistication frontier. The observation from Nguyen et al. [22] that CPU and memory metrics are inadequate signals for content-inspection workloads is directly supported by the present results: SDLB's use of ICAP-layer health scores, rather than resource utilisation, as the input to its adaptive decision engine was what allowed detection of situations that a pure resource-metric system would have missed entirely, for example the ICAP penalty on version A scenario with a health score of 75.

Compared with Hahn et al.'s [14] finding that service mesh security requires additional admission-level controls, SDLB's design explicitly delegates admission-level control to EAPE and focuses on routing-level control only. This separation aligns with the architectural decomposition those authors advocate, and the present experiments confirm that the routing-level control is effective within its own scope.

3.9.4 Implications for Cloud-Native Security Infrastructure

Three broader implications emerge from the results. First, the demonstrated feasibility of environment-aware routing suggests that the one-size-fits-all configuration patterns common in present-day Kubernetes deployments are leaving operational value on the table. Organisations that operate multiple environment tiers in a single cluster, which is most organisations, stand to benefit from tier-differentiated policies, and the tooling to express and enforce those policies is now demonstrably tractable.

Second, the 7.99 per cent routing overhead is low enough that inline security enforcement at the routing layer is a viable architectural pattern rather than a theoretical ideal. Historically, security architects have been reluctant to place enforcement points on the request hot path for fear of latency impact. The present results indicate that such reluctance is, for Istio-based routing at least, overcautious: inline enforcement is feasible at costs well within production-grade budgets.

Third, the compliance mapping work (Table 2.4) demonstrates that routing decisions and compliance audit events can be unified in a single data flow. Rather than producing routing logs and compliance logs as separate artefacts, SDLB produces a single structured event stream that serves both purposes. This reduces the integration burden on downstream audit systems and eliminates the category of bugs in which the routing logs and the compliance logs disagree about what happened.

3.9.5 Future Research Directions

Five directions for future research suggest themselves. First, data-driven optimisation of the adaptive decision engine's threshold constants. The 8 per cent / 3 per cent / 20 per cent / 60 s thresholds were chosen by manual tuning against observed traffic profiles; a principled optimisation, for example through a cost function that balances false-positive and false-negative detections against observed traffic distributions, would likely yield threshold values that are optimal across a wider range of deployment conditions.

Second, extension to cross-cluster federation. The present SDLB operates within a single Kubernetes cluster. Multi-cluster deployments, which are increasingly common in geographically distributed enterprises, introduce routing decisions that span clusters and require coordination between federated SDLB controllers. A federation protocol for sharing health state and traffic trends across clusters is a natural next research target.

Third, incorporation of machine-learning-driven prediction into the decision engine. The present engine uses simple moving averages and threshold rules; richer time-series models could produce earlier and more accurate predictions of traffic spikes, reducing the lag between spike onset and spread-mode entry. Care would be needed to preserve interpretability, as the current engine's textual reason strings are a significant operator feature, but interpretable ML approaches such as decision trees could be adopted without sacrificing this property.

Fourth, extension of the compliance catalogue beyond PCI-DSS and CIS. GDPR, HIPAA, and emerging cloud-specific frameworks such as FedRAMP each have audit-logging requirements that could be mapped to SDLB events. A compliance-catalogue plugin architecture would allow operators to activate the mappings relevant to their regulatory environment.

Fifth, evaluation on production-grade managed Kubernetes distributions such as EKS, GKE, and AKS. The present evaluation is confined to Minikube, and while Minikube is a faithful approximation, managed distributions introduce additional control-plane constraints and observability platforms that would merit direct evaluation. This is the most straightforward of the five directions in terms of technical content, but would produce additional confidence that the results generalise beyond the constrained experimental environment.

CHAPTER 4 — CONCLUSION

Chapter 4 concludes the thesis by synthesising the contributions of the Service Discovery and Load Balancing component, mapping each observed result back to the research objectives stated in Section 1.5, enumerating the practical implications of the work, acknowledging the remaining limitations, and closing with reflections on the broader significance of the research.

4.1 Summary of Research Contributions

Three technical artefacts and one methodological result have been produced by this thesis. The first technical contribution is an environment-aware routing framework that unifies service discovery, policy-driven load balancing, and compliance audit logging within a single Kubernetes-native controller. The framework is characterised by three distinct base policies, namely strict health-gated round-robin for Production, weighted 50/30/20 for Staging, and permissive single-instance routing for Development, each driven deterministically from environment classification signals produced by an upstream policy engine. This tier-differentiated design is, to the author’s knowledge, the first empirically evaluated implementation of environment-aware routing for malware inspection in Kubernetes; prior work has either applied uniform policies or relied on manual per-tier configuration.

The second technical contribution is the adaptive routing decision engine that layers traffic-trend hysteresis, per-instance ICAP health penalties, cooldown-based oscillation suppression, and predictive spread-mode transitions on top of the base policies. The engine’s novelty lies in the health-penalty-weighted effective-rate formula, $\text{effective_rate} = \text{observed_rate} \times (1 + (\text{floor} - \text{health}) / \text{floor})$, which smoothly transforms per-instance health signals into routing preferences without producing discrete threshold transitions. The engine was validated against eight canonical scenarios that exercise every branch of its decision hierarchy, and the live controller produced the reference decision in every case.

The third technical contribution is a complete compliance mapping between SDLB routing events and two widely used frameworks: PCI-DSS v4.0 [9] and the CIS Kubernetes Benchmark v1.9 [8]. Six categories of routing events are mapped to specific control identifiers (Table 2.4), and every event emitted by the live controller carries the corresponding annotations as part of its structured output. This mapping allows the same routing-event stream to serve both operational observability and compliance audit purposes, reducing integration burden and eliminating a category of inconsistency bug.

The methodological contribution is the testbed methodology used for evaluation, which extends the methodology of Saleh Sedghpour et al. [13] by adding tier-differentiated workloads, ICAP-specific failure injection, and compliance-event verification. The experimental setup is deterministic and reproducible on any Kubernetes cluster of comparable size, making the results amenable to independent replication and comparison.

4.2 Achievement of Research Objectives

The five research objectives enumerated in Section 1.5 each have a quantitative evaluation criterion, and the observed results in Chapter 3 demonstrably satisfy each criterion.

Objective 1, Environment-Aware Routing Correctness, was evaluated against inter-instance variance, weight deviation, and primary-instance share thresholds. The measured

values of 0.7 per cent, ± 1.0 per cent, and 97.6 per cent are each within the corresponding acceptable thresholds (< 2.0 per cent, ± 2.0 per cent, > 95.0 per cent). The objective is met.

Objective 2, Health-Driven Failover, was evaluated against mean detection time (< 35 s), mean redistribution time (< 5 s), and request loss during failover (0). The measured values of 29.7 s, 3.9 s, and 0 requests respectively satisfy every criterion. The objective is met.

Objective 3, Low Routing Overhead, was evaluated against a 10 per cent overhead threshold. The measured 7.99 per cent overhead, classified as Good, is comfortably below the threshold. The objective is met.

Objective 4, Adaptive Decision Engine, was evaluated against correctness across eight canonical scenarios. Every scenario produced the reference decision and the reference reasoning. The objective is met.

Objective 5, Compliance Mapping, was evaluated against complete mapping coverage for the six categories of routing events produced by the controller. Table 2.4 enumerates the full mapping, and every event emitted during the experiments in Chapter 3 carried the corresponding annotations. The objective is met. In summary, all five research objectives have been met, spanning routing correctness, failover behaviour, system overhead, adaptive decision-making, and compliance, which are exactly the five concerns that the research problem identified as operationally critical for a production-grade routing layer.

4.3 Practical Implications

The work has several practical implications for organisations operating Kubernetes clusters that perform inline content inspection. First, environment-aware routing is technically feasible and operationally beneficial. The 20.8 per cent latency improvement achieved in Development, combined with unaltered Production security properties, demonstrates that tier-differentiated routing yields meaningful benefits for developer productivity without compromising the security regime of production workloads. Organisations currently running uniform routing configurations should consider whether tier-differentiated policies would yield similar benefits in their own environments.

Second, routing-layer inspection enforcement is viable at low overhead. The 7.99 per cent mean overhead measured here is well within the budget most security architects allocate for inline enforcement, which historically has been treated as a costly architectural concession. The overhead figure argues for broader adoption of routing-layer security enforcement rather than reliance on perimeter-only enforcement approaches, a conclusion particularly relevant to organisations implementing Zero Trust architectures aligned with NIST SP 800-207 [25].

Third, compliance-annotated events reduce audit integration costs. By producing a single event stream that serves both operational and compliance purposes, SDLB eliminates the need for a separate compliance translation layer in the audit pipeline. Organisations subject to PCI-DSS or CIS Benchmark obligations should find the integration of such a component straightforward, and the risk of log-to-audit disagreement is structurally eliminated. Fourth, the open-source nature of the underlying technologies, including Kubernetes, Istio, c-icap, and ClamAV, means that the architecture is not tied to any particular vendor. Organisations with heterogeneous cloud strategies can deploy SDLB across any Kubernetes distribution without vendor-specific adaptations. Fifth, the commercialisation pathway outlined in Section 2.9 indicates that the architecture has credible commercial applicability, with a

defensible open-core licensing model targeting regulated enterprises in financial services, healthcare, and government sectors.

4.4 Limitations and Future Work

Four limitations of the present work merit explicit acknowledgement. First, the evaluation was conducted on Minikube rather than a managed Kubernetes distribution. Although Minikube is a faithful approximation, production-grade distributions such as EKS, GKE, and AKS introduce additional control-plane features and constraints that could alter the observed results. Future work should replicate the experiments on at least one managed distribution to establish cross-platform validity. The replication is straightforward in principle, as the controller and its dependencies are portable, but the experimental setup and traffic generation require rebuilding on each target distribution.

Second, the number of ICAP replicas tested is capped at three. Real-world deployments may run tens or hundreds of inspection instances, and the scaling behaviour of SDLB beyond three instances is not empirically established. The architectural analysis suggests that scaling should remain linear until other bottlenecks, such as the controller's reconciliation rate, the Istiod push rate, and the Kubernetes API server's write capacity, saturate, but this is an engineering projection rather than a measured result. A follow-up study scaling to 10 and 50 replicas is a natural next step.

Third, the threshold constants in the adaptive decision engine were chosen by manual tuning against observed traffic profiles. The 8 per cent entry, 3 per cent exit, 20 per cent minimum change, and 60-second cooldown thresholds are reasonable starting points but are not demonstrably optimal across all deployment conditions. A principled optimisation study, for example a cost-function-based optimisation over a range of traffic patterns, would likely yield threshold values that are more robustly performant. Fourth, the traffic generation methodology produces scripted inspection workloads that approximate but do not faithfully reproduce real-world traffic; production traffic exhibits diurnal variation, long-tailed payload-size distributions, and workload-specific patterns that are absent from the present experiments. These limitations all point toward the future research directions identified in Section 3.9.5.

4.5 Closing Remarks

This thesis has presented the design, implementation, and empirical evaluation of the Service Discovery and Load Balancing component of the CAPSLock Kubernetes security platform. The work addresses a real operational gap, namely the absence of environment-aware routing for malware inspection in Kubernetes, and demonstrates, by construction and by measurement, that the gap can be filled with a low-overhead, auditable, and commercially viable component.

The broader significance of the work extends beyond its immediate technical contributions. The cloud-native ecosystem has matured to the point where the naïve uniformity of early-generation configurations is increasingly untenable, and tools that automate tier-differentiated policy application are becoming essential rather than optional. SDLB is one such tool. The environment-aware routing pattern it implements is generalisable to other policy-sensitive concerns, including access control, observability sampling, and chaos-engineering injection, and the compliance-mapping approach it exhibits is applicable wherever a data-plane component needs to produce audit-ready output.

The CAPSLock platform of which SDLB is one part represents a more ambitious undertaking: a four-component security gateway that addresses promotion control, environment classification, inspection traffic routing, and content scanning in a unified architecture. The SDLB component described in this thesis is one concrete demonstration that each of these components can be built to a production-grade standard within the constraints of an undergraduate research project. The empirical results reported in Chapter 3, all twelve PASS across the consolidated benchmark summary, are the quantitative realisation of that demonstration.

The work reported here is intended to be of use both to academic researchers pursuing further investigation of environment-aware cloud-native security and to practitioners charged with operating regulated Kubernetes deployments. The architecture is documented, the thresholds are published, the compliance mappings are enumerated, and the experimental methodology is reproducible. The foundations for further work are in place.

REFERENCES

- [1] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security: A Systematization of Knowledge Related to Kubernetes Security Practices," in *Proc. IEEE Secure Development (SecDev)*, Atlanta, GA, USA, 2020, pp. 58–64, doi: 10.1109/SecDev45635.2020.00025.
- [2] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, Art. 99, Jul. 2023, doi: 10.1145/3579639.
- [3] S. Sultan, I. Ahmad, and T. Dimitriou, "Container Security: Issues, Challenges, and the Road Ahead," *IEEE Access*, vol. 7, pp. 52976–52996, 2019, doi: 10.1109/ACCESS.2019.2911732.
- [4] F. Minna, A. Blaise, F. Rebecchi, B. Chandrasekaran, and F. Massacci, "Understanding the Security Implications of Kubernetes Networking," *IEEE Security & Privacy*, vol. 19, no. 5, pp. 46–56, Sep./Oct. 2021, doi: 10.1109/MSEC.2021.3094726.
- [5] A. Sissodiya et al., "Formal Verification for Preventing Misconfigured Access Policies in Kubernetes Clusters," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3597504.
- [6] C. Cesarano and R. Natella, "KubeFence: Security Hardening of the Kubernetes Attack Surface," in *Proc. 55th Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*, 2025, pp. 497–510, doi: 10.1109/DSN64029.2025.00054.
- [7] M. Souppaya, J. Morello, and K. Scarfone, *Application Container Security Guide*, NIST Special Publication 800-190, National Institute of Standards and Technology, Sep. 2017, doi: 10.6028/NIST.SP.800-190.
- [8] Center for Internet Security, *CIS Kubernetes Benchmark, Version 1.9*, CIS, 2024. [Online]. Available: <https://www.cisecurity.org/benchmark/kubernetes>
- [9] Payment Card Industry Security Standards Council, *Payment Card Industry Data Security Standard, Version 4.0, PCI SSC*, Mar. 2022. [Online]. Available: <https://www.pcisecuritystandards.org>
- [10] L. Rice, *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. Sebastopol, CA, USA: O'Reilly Media, 2020. ISBN: 978-1492056706.
- [11] Open Policy Agent Project, "Open Policy Agent: Policy-based Control for Cloud Native Environments," *Cloud Native Computing Foundation*, 2021. [Online]. Available: <https://www.openpolicyagent.org>
- [12] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, "Exploring Security Practices in Infrastructure as Code: An Empirical Study," *Empirical Softw. Eng.*, vol. 29, Art. 182, 2024, doi: 10.1007/s10664-024-10610-0.
- [13] M. R. Saleh Sedghpour, C. Klein, and J. Tordsson, "An Empirical Study of Service Mesh Traffic Management Policies for Microservices," in *Proc. ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, Beijing, China, 2022, pp. 17–27, doi: 10.1145/3489525.3511686.
- [14] D. A. Hahn, D. Davidson, and A. G. Bardas, "MisMesh: Security Issues and Challenges in Service Meshes," in *Security and Privacy in Communication Networks (SecureComm 2020)*, *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, vol. 335, Cham: Springer, 2020, pp. 140–151, doi: 10.1007/978-3-030-63086-7_8.
- [15] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, "Service Mesh: Challenges, State of the Art, and Future Research Opportunities," in *Proc. IEEE Int. Conf. on Service-Oriented System Engineering (SOSE)*, 2019, pp. 122–127, doi: 10.1109/SOSE.2019.00026.
- [16] R. Chandramouli and Z. Butcher, *Building Secure Microservices-based Applications Using Service-Mesh Architecture*, NIST Special Publication 800-204A, National Institute of Standards and Technology, May 2020, doi: 10.6028/NIST.SP.800-204A.
- [17] J. Elson and A. Cerpa, *Internet Content Adaptation Protocol (ICAP)*, IETF RFC 3507, Apr. 2003. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc3507>
- [18] A. Y. Wong, E. G. Chekole, M. Ochoa, and J. Zhou, "On the Security of Containers: Threat Modeling, Attack Analysis, and Mitigation Strategies," *Computers & Security*, vol. 135, Art. 103490, Dec. 2023, doi: 10.1016/j.cose.2023.103490.

- [19] M. Hausenblas and S. Schimanski, *Programming Kubernetes: Developing Cloud-Native Applications*. Sebastopol, CA, USA: O'Reilly Media, Jul. 2019. ISBN: 978-1492047100.
- [20] J. Dobies and J. Wood, *Kubernetes Operators: Automating the Container Orchestration Platform*. Sebastopol, CA, USA: O'Reilly Media, Mar. 2020. ISBN: 978-1492048046.
- [21] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, May 2016, doi: 10.1145/2898442.2898444.
- [22] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal Pod Autoscaling in Kubernetes for Elastic Container Orchestration," *Sensors*, vol. 20, no. 16, Art. 4621, Aug. 2020, doi: 10.3390/s20164621.
- [23] CNCF TAG Security, *Cloud Native Security Whitepaper, Version 2*, Cloud Native Computing Foundation, May 2022. [Online]. Available: <https://github.com/cncf/tag-security>
- [24] D. D'Silva and D. D. Ambawade, "Building A Zero Trust Architecture Using Kubernetes," in *Proc. 6th Int. Conf. for Convergence in Technology (I2CT)*, 2021, pp. 1–8, doi: 10.1109/I2CT51068.2021.9418203.
- [25] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, *Zero Trust Architecture*, NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020, doi: 10.6028/NIST.SP.800-207.
- [26] B. Burns, J. Beda, K. Hightower, and L. Evenson, *Kubernetes: Up and Running: Dive into the Future of Infrastructure*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022. ISBN: 978-1098110208.
- [27] Istio Project, "Istio Architecture," Istio Documentation, 2024. [Online]. Available: <https://istio.io/latest/docs/ops/deployment/architecture/>
- [28] B. Liu, C. Zhang, B. Li, and J. Tu, "Health-Aware Load Balancing for Microservices in Service Mesh Architectures," in *Proc. IEEE Int. Conf. on Cloud Computing (CLOUD)*, 2021, pp. 361–369, doi: 10.1109/CLOUD53861.2021.00054.
- [29] S. S. Gill et al., "Transformative Effects of IoT, Blockchain and Artificial Intelligence on Cloud Computing: Evolution, Vision, Trends and Open Challenges," *Internet of Things*, vol. 8, Art. 100118, 2019, doi: 10.1016/j.iot.2019.100118.
- [30] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, Mar. 2014.